HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

HUST

SOICT

PROJECT REPORT

MEDORA: Integrated Clinic Management and Electronic Medical Records System

Course: System Design
Class ID: 175882
Supervisor: Ph.D Nguyen Huu Duc

Member	Student ID
Nguyen Tran Minh Tri	202400116
Le Tuan Hung	202416800
Nguyen Khac Hoang Phuc	202416818
Nguyen Hoang Quan	202416738
Nguyen Duc Dung	202416681

Ha Noi, 2026

Abstract

MEDORA is a proposed integrated clinic-management and electronic medical-record (EMR) system for outpatient clinics and small healthcare facilities. It addresses workflow fragmentation across patient registration, appointment scheduling, check-in, clinical documentation, prescriptions, service execution, billing and operational reporting. The proposed solution centres on a shared patient and encounter record, explicit workflow states, role-based access control and an audit trail for sensitive actions. This report develops the business model, domain concepts, business rules, functional and non-functional requirements, use-case model and traceability needed to guide the subsequent system design and evaluation. The scope is deliberately limited to internal clinic operations; clinical decision-making, insurer settlement, national interoperability and production compliance certification remain outside the current project boundary.

Keywords: electronic medical records, clinic management, outpatient workflow, role-based access control, system analysis and design.

Contents

List of Figures 5

List of Tables 6

1 Introduction and Problem Definition 7
- 1.1 Background and Existing System . . . 7
- 1.2 Problem Statement and Motivation . . . 8
- 1.3 Project Objectives and Success Criteria . . . 9
- 1.4 System Scope, Assumptions, and Constraints . . . 9
- 1.5 Proposed Solution and System Overview . . . 10

2 Business Analysis and Domain Modeling 11
- 2.1 Business Context and Operational Overview . . . 11
- 2.2 Stakeholders and Responsibilities . . . 12
- 2.3 Business Workflow and Controls . . . 13
- 2.4 Business Rules . . . 16
- 2.5 Domain Model and Core Entities . . . 16

3 System Requirements Analysis 19
- 3.1 Functional Decomposition and Package Structure . . . 19
- 3.2 Actors and Overall Use Case Model . . . 19
- 3.3 Use Case Diagrams by Functional Group . . . 19
  - 3.3.1 Patient Management . . . 20
  - 3.3.2 Appointment, Check-in and Queue Management . . . 21
  - 3.3.3 Medical Examination and Encounter Management . . . 23
  - 3.3.4 Prescription, Billing and Governance . . . 26
- 3.4 Detailed Use Case Specifications . . . 28
  - 3.4.1 UC-021 Book Appointment . . . 28
  - 3.4.2 UC-027 Check In Patient . . . 29
  - 3.4.3 UC-037/UC-038 Sign and Amend a Clinical Note . . . 29
  - 3.4.4 UC-048 Validate and Issue Prescription . . . 29
  - 3.4.5 UC-039/UC-040 Service Order, Allocation and Result . . . 30
  - 3.4.6 UC-063/UC-065 Record Payment and Refund . . . 30
- 3.5 Functional Requirements . . . 31
- 3.6 Non-functional Requirements . . . 31
- 3.7 Role-based Access Control Matrix . . . 32
- 3.8 Requirements Traceability and Use Case Validation . . . 32

4 System Design and Structural Modeling 36
- 4.1 System Architecture . . . 36
  - 4.1.1 Logical Architecture . . . 36

4.1.2 Deployment Architecture . . . 36
4.2 Layered Design and Design Patterns . . . 36
4.2.1 Presentation, Business Logic, and Data Access Layers . . . 36
4.2.2 API and Web Service Design . . . 36
4.3 Database Design and Data Integrity . . . 36
4.3.1 Entity–Relationship Diagram . . . 36
4.3.2 Relational Schema and Normalization . . . 36
4.3.3 Persistent Classes and Database Mapping . . . 36
4.3.4 Keys, Constraints, Data Types, and Multiplicities . . . 36
4.4 Structural UML Diagrams . . . 36
4.4.1 Overall Class Diagram . . . 36
4.4.2 Class Diagrams by Use Case Group . . . 36
4.4.3 Package Diagram . . . 36
4.4.4 Component Diagram . . . 36
4.4.5 Deployment Diagram . . . 36
4.4.6 Object Diagram (if required) . . . 36
4.5 Use Case-oriented Design . . . 36
4.5.1 Participating Classes and Their Responsibilities . . . 36
4.5.2 Collaboration Between System Components . . . 36

5 Behavioral Modeling and System Implementation 36
5.1 Behavioral UML Diagrams . . . 36
5.1.1 Sequence Diagrams for Key Use Cases . . . 37
5.1.2 Activity Diagrams for Complex Workflows . . . 37
5.1.3 State Machine Diagrams for Lifecycle-driven Entities . . . 37
5.2 Source Code Organization and Module Structure . . . 48
5.3 Implementation of Core Functionalities . . . 49
5.4 Development and Deployment Environment . . . 50
5.5 Proposed User Interfaces and API Evidence . . . 51

6 System Testing and Evaluation 51
6.1 Testing Strategy and Test Environment . . . 51
6.2 Test Scenarios and Test Case Design . . . 52
6.3 Functional Testing and Use Case Coverage . . . 53
6.4 Exception Handling, Authorization, and Data Integrity Testing . . . 53
6.5 Test Execution Results and Evidence . . . 54
6.6 Evaluation Against Functional and Non-functional Requirements . . . 54
6.7 Limitations and Outstanding Issues . . . 55

7 Conclusion and Future Work 55
7.1 Summary of Achievements . . . 55
7.2 Limitations of the Current System . . . 55

7.3 Future Improvements and Extensions 55

A Detailed Use Case Specifications **56**
A.1 UC-021 Book Appointment 56
A.2 UC-027 Check In Patient 57
A.3 UC-037/UC-038 Sign and Amend Clinical Note 58
A.4 UC-048 Validate and Issue Prescription 59
A.5 UC-039/UC-040 Schedule and Execute Service Order 59
A.6 UC-063/UC-065 Record Payment and Refund 60

B Supplementary UML Diagrams **61**

C API Specifications and Data Schemas **62**
C.1 Representative Appointment Data 62
C.2 Representative Encounter Data 62
C.3 Representative Audit Event Data 63

D Detailed Test Cases and Execution Results **63**

References **65**

List of Figures

1 Target business workflow for a standard outpatient journey 14
2 Appointment and check-in workflow with alternative and exception paths 15
3 Core clinical conceptual domain model 17
6 Patient-management use cases 20
7 Appointment-management use cases 21
8 Check-in and queue-management use cases 22
9 Screening and consultation use cases 23
10 Clinical-note lifecycle and encounter-closure use cases 24
11 Service-order and execution use cases 25
12 Prescription and medication-safety use cases 26
13 Billing and payment use cases 27
14 Administration, reporting and governance use cases 28
4 Functional decomposition of MEDORA 34
5 Overall use-case model of MEDORA 35
15 System sequence for appointment booking and availability validation 38
16 System sequence for check-in, encounter creation, and queue assignment 39
17 System sequence for signing a clinical note and appending a correction 40
18 System sequence for configured prescription safety checks and issue 41
19 System sequence for resource allocation and service-order execution 42
20 System sequence for payment and separately authorized refund events 43
21 Activity model for service-order allocation and execution 44
22 Activity model for encounter closure and independent financial settlement 45
23 Proposed appointment lifecycle and terminal outcomes 46
24 Proposed encounter lifecycle from check-in to clinical completion 46
25 Clinical-note version lifecycle with immutable signed versions 47
26 Proposed procedure-order lifecycle with allocation and execution outcomes 47
27 Proposed invoice lifecycle for unpaid, partial-payment, and paid states 48

List of Tables

1 MEDORA product definition and value proposition 10
2 Business baseline and target operating model 12
3 Stakeholder responsibilities and authority boundaries 12
4 Role-to-activity mapping and interaction boundaries 13
5 Major business processes, outcomes and controlled alternatives 16
6 Core business rules 16
7 Core entity glossary 18
8 Core domain relationships 18
9 Business-to-system traceability 18
16 Functional requirements catalogue 31
17 Non-functional requirements catalogue 31
19 Requirements traceability and planned verification 32
18 Target role-based access-control matrix 34
20 Implementation and deployment evidence available in the repository 49
21 Proposed implementation slices and acceptance invariants 50
22 Proposed development and deployment environment matrix 50
23 Planned verification levels and present evidence boundary 52
24 Planned test cases and current execution status 52
25 Representative functional and quality-requirement test coverage 53
26 Test execution summary at report time 54
27 Evaluation criteria and evidence status 54
28 UML source and rendered-figure inventory 61
29 Detailed test procedures and expected outcomes 63

1 Introduction and Problem Definition

1.1 Background and Existing System

Healthcare delivery is becoming a coordination problem as much as a clinical one. Across countries, providers are expected to offer safer, more continuous and more patient-centred care while dealing with rising demand, constrained staff capacity and increasingly complex information flows. The World Health Organization (WHO) estimates that approximately one in ten patients is harmed during health care and that more than half of this harm is preventable; medication-related harm accounts for a substantial share of the preventable cases [World Health Organization, 2023]. These figures do not imply that software can replace clinical judgement. They do, however, show why reliable identification, complete records, clear hand-offs and traceable actions are system-level concerns rather than administrative details.

Digital health is therefore moving from a collection of isolated applications towards an integrated operating capability. The WHO describes digital health as a means of aligning organizational, human and technological resources around better health outcomes, not simply as the deployment of another application [World Health Organization, 2021b]. Yet digitalisation can reproduce the same fragmentation when applications are implemented as disconnected, supply-driven tools; WHO identifies integration over isolation, data governance and protection of privacy as essential conditions for useful digital transformation [World Health Organization, 2021a; World Health Organization Alliance for Health Policy and Systems Research, 2024]. In a clinic, this distinction is important: an appointment calendar, a patient database and a billing screen may each work in isolation while the overall journey still depends on repeated data entry, informal communication and manual reconciliation. The result is a fragile process in which a patient can be correctly registered in one place but remain invisible to the next person who must act.

The persistence of legacy workflows is not unique to one country. The OECD reports that 75% of global fax traffic is used for medical services and that evidence-based innovations can take as long as 17 years to become routine practice, illustrating the distance between available technology and dependable day-to-day adoption [Organisation for Economic Co-operation and Development, 2023]. For a small clinic, the implication is practical: the priority is not to buy the most sophisticated technology, but to design a coherent system that staff can use consistently and that can exchange trustworthy data when the organization grows.

The Vietnamese context makes this challenge particularly tangible. Viet Nam entered the ageing phase in 2011; people aged 60 and over represented 11.9% of the population in 2019, and this share is projected to exceed 25% by 2050 [United Nations Population Fund Viet Nam, 2021]. An ageing population increases the need for repeated visits, longitudinal clinical histories and coordination across services. At the point of care, operational friction is already measurable. In a study of 7,931 outpatient visits at Thong Nhat Hospital, the mean total waiting time was 104.1 minutes, with departments handling roughly 150–250 patients per day while clinic capacity and staffing remained comparatively stable [Nguyen et al., 2018]. A separate study across Vietnamese health facilities reported average waits of 32.58 minutes at district hospitals and 11.58 minutes at commune health centres before patients met clinical staff [Nguyen et al., 2020]. These are not merely comfort issues: long queues

consume staff time, obscure workload, increase opportunities for transcription errors and weaken patients' confidence in the service.

1.2 Problem Statement and Motivation

The need for an electronic medical-record system is visible in three connected indicators. First, outpatient flow is slow: a study of 7,931 visits at a Vietnamese national hospital reported a mean total waiting time of 104.1 minutes, while departments handled approximately 150–250 patients per day without proportional capacity growth [Nguyen et al., 2018]. A broader study reported average waits of 32.58 minutes at district hospitals and 11.58 minutes at commune health centres before patients met clinical staff [Nguyen et al., 2020]. Second, patient-safety risk remains material: the WHO estimates that one in ten patients is harmed in health care and that more than half of this harm is preventable [World Health Organization, 2023]. Third, medication errors alone are estimated to cost about US$42 billion globally each year [World Health Organization, 2017].

These figures do not prove that every clinic has the same performance. They establish why the workflow deserves system-level attention. When identity, appointments, clinical notes, orders and payment are split across paper forms, spreadsheets or isolated applications, the following problems become likely:

- duplicate patient records and repeated demographic entry;
- appointment, room or staff conflicts that are discovered too late;
- incomplete visibility of allergies, previous encounters, current medication or open orders;
- unclear responsibility when an order moves from prescription to execution;
- silent overwriting of signed information and weak evidence of who changed it; and
- limited management visibility into waiting, workload, service progress and payment status.

WHO identifies fragmented data governance and weak exchange mechanisms as barriers to timely, high-quality use of health data [World Health Organization, 2021a]. MEDORA addresses this coordination gap by providing one patient identity, one encounter context and explicit workflow states across authorized roles. Its "smart" behaviour is deliberately bounded: the system applies rules, warnings, validation and audit trails at the right step, while diagnosis and treatment decisions remain with qualified clinicians.

The architecture is part of the motivation. Separating presentation, application rules, domain entities and persistence prevents a new screen from bypassing clinical or access-control rules. Explicit services and state transitions make double-booking prevention, signed-record amendment, authorization and auditability testable. The result is a system that is safer to operate today and easier to extend tomorrow, rather than a set of screens that merely digitizes existing fragmentation.

1.3 Project Objectives and Success Criteria

This project proposes MEDORA, an integrated clinic-management and electronic medical-record system for a clinic or small healthcare facility. Its objective is to turn a fragmented sequence of tasks into a controlled, observable and reusable workflow. The project pursues five linked outcomes:

- one stable patient identity and one longitudinal record across visits;
- coordinated appointments, queues, rooms and staff;
- structured support for examination, diagnosis, prescriptions and service orders;
- auditable handling of sensitive changes and role-based access to health data; and
- operational dashboards and reports that support evidence-based management.

Success is specified using SMART criteria so that the prototype can be evaluated without overstating clinical impact. During the final test cycle, against a documented baseline workload, MEDORA will:

- reduce median registration-to-completion time by at least 25% for the representative outpatient scenario;
- accept zero double-bookings for the same doctor, room or constrained resource across 100% of the concurrency test cases;
- achieve at least 95% completion of configured mandatory encounter fields across the tested completed visits;
- record an actor, timestamp, action and affected entity for 100% of tested sensitive events, including clinical amendments, prescription changes, exports and permission changes; and
- demonstrate that 100% of negative authorization test cases are rejected at the API/server boundary, while the principal happy-path workflow completes successfully in the agreed test environment.

Each target is specific to a defined workflow, measurable through test evidence, achievable for the course-project scope, relevant to the stated pain points and time-bound to the final evaluation cycle. The figures are prototype acceptance targets, not claims of production performance; a real deployment would first establish a site-specific baseline and then revalidate them with clinical and operational stakeholders.

1.4 System Scope, Assumptions, and Constraints

MEDORA is an internal web system for a clinic or small healthcare facility. It supports the outpatient journey from patient identification and appointment or walk-in reception to check-in, screening, consultation, prescription or service order, execution, encounter closure and payment status.
Receptionists manage identity, appointments and queues; nurses record screening; doctors own clinical decisions; technicians and medicine staff record actual execution; cashiers record financial events;

managers, administrators and auditors supervise operations, configuration and accountability. The system preserves the difference between planned and completed work: an appointment is not an encounter, a prescription is not a medicine event, an order is not a result, and an invoice is not a payment. The MVP focuses on internal operations, basic billing, reporting and governance. Patient self-service, external diagnostic systems, insurer settlement and advanced inventory are later-stage extensions. Every sensitive action is constrained by role, scope and workflow state; signed notes are amended rather than overwritten, and concurrent booking must not create a conflict.

1.5 Proposed Solution and System Overview

MEDORA is a clinic operating system built around one shared patient and encounter record, not a collection of unrelated modules. Its central unit of work is the outpatient encounter. A Patient may have many Appointments and Encounters; an Encounter is the controlled context for clinical notes, vital signs, diagnoses, prescriptions and procedure orders; an order may consume a scheduled ResourceBooking and later receive an execution result; and an Invoice and Payment record the financial consequence without changing the clinical history. Attachments, notifications and AuditLogs are linked to the relevant business object rather than stored as anonymous files or messages.

Table 1 defines the product by capability. Each row states the function, its primary users and the operational value it is expected to create. This makes the product boundary concrete before the report describes its technical realization: MEDORA is an internal clinic-management and electronic medical-record product that coordinates people, information, resources and decisions throughout an outpatient episode.

Table 1: MEDORA product definition and value proposition

Function	Primary users	What the system does	Value delivered
Patient identity and record management	Reception, nurses, doctors	Search/register patients, flag duplicates and link history, allergies, documents and encounters.	One longitudinal record and safer identification.
Appointment, schedule and queue management	Reception, managers; patient portal (Phase 2)	Book, reschedule, cancel, check in and assign queues against current availability.	Visible workload and fewer coordination conflicts.
Clinical examination and EMR	Nurses, doctors	Capture screening, vital signs, consultation, diagnosis and follow-up in the Encounter.	Continuous, structured clinical context.
Prescription and medication safety	Doctors, medicine staff	Issue prescriptions with directions and configured safety checks; retain amendment and medicine-event history.	Safer hand-offs between prescribing and dispensing.
Procedure, service and resource coordination	Doctors, nurses, technicians	Create orders, allocate constrained resources and record execution/result.	Prevents double-booking and links decision to outcome.

Function	Primary users	What the system does	Value delivered
Billing and payment status	Cashiers, managers	Create invoices and retain payment/refund history separately from clinical content.	Clear financial accountability and reconciliation.
Administration and role-based access	Administrators, managers, auditors	Manage users, permissions, catalogues, schedules and scope.	Least privilege and adaptable operations.
Dashboards, reports and audit trail	Managers, auditors	Present operational indicators, controlled export and sensitive-event history.	Evidence for decisions and investigation.
Attachments and notifications	Assigned staff; patient portal (Phase 2)	Store controlled documents and send privacy-aware operational notifications.	Reliable follow-up with lower privacy exposure.

The solution uses four cooperating layers:

- **Presentation:** role-specific workspaces for reception, nursing, consultation, service execution, cashier operations and management.
- **Application:** use cases such as registration, booking, check-in, consultation, prescription, service scheduling, result recording, encounter closure and payment.
- **Domain:** rules for availability, permissions, state transitions, signed-record amendments and clinical warnings.
- **Data:** persistent patient, encounter, order, resource, invoice and audit entities with integrity constraints and history.

Each workflow step has an explicit state and accountable actor. Backend authorisation, controlled attachments and audit history protect the record while the modular structure leaves a clear path for later portal, diagnostic and interoperability extensions.

2 Business Analysis and Domain Modeling

2.1 Business Context and Operational Overview

MEDORA is an internal web system for an outpatient clinic or small healthcare facility. It coordinates one outpatient episode from patient identification and appointment handling to consultation, service execution, payment status and operational reporting. Its central design principle is continuity: every authorised role works from the same patient and encounter context rather than from disconnected records.

Four distinctions protect that continuity. An Appointment reserves time, while an Encounter represents the visit that actually occurs. A prescription is a clinical decision; dispensing or administration is a later, accountable event. A service order is not an execution result. An Invoice records charges,

whereas a Payment or refund records a financial transaction. These distinctions prevent the system from reporting planned work as completed work.

The MVP covers internal clinic operations, basic billing, reporting and governance. Patient self-service, external laboratory or imaging interfaces, insurer settlement and other integrations are later-stage capabilities.

Business area	Target operating model
Patient identity	Search before registration; flag possible duplicates; preserve merge history.
Visit coordination	Treat appointment, check-in, encounter and queue placement as explicit states.
Clinical care	Keep screening, documentation, diagnosis, prescription and orders in one encounter context.
Execution and finance	Record allocation, service execution, invoice and payment as separate events.
Governance	Apply scoped access, controlled export and auditable sensitive actions.

Table 2: Business baseline and target operating model

2.2 Stakeholders and Responsibilities

Table 3 defines the roles that participate in the target workflow. Access is always restricted by role, facility, assignment and workflow state; managerial responsibility does not create unrestricted clinical-edit access.

Role	Primary responsibility	Authority boundary
Receptionist	Identify/register patients; manage appointments, arrivals and queues.	Administrative and operational data only.
Nurse	Record screening, vital signs and assigned care activity.	Cannot alter diagnosis or signed prescriptions.
Doctor	Consult, document, diagnose, prescribe, order services and close encounters.	Owns clinical decisions for assigned encounters.
Technician / Pharmacist	Execute assigned service work or record the medicine event.	Records execution; does not create the clinical decision.
Cashier	Create eligible invoices and record payments or refunds.	Cannot edit clinical content.
Manager	Monitor capacity, services, revenue and reports.	Uses scoped operational information.
Administrator / Auditor	Configure access/catalogues or review audit evidence.	Administration and audit are separate from clinical authority.
Patient (Phase 2)	Request appointments and view released documents.	Accesses only verified, explicitly released information.

Table 3: Stakeholder responsibilities and authority boundaries

Actor	Main system interaction	Boundary
Receptionist	Patient search, registration, appointment, check-in and queue.	No clinical decision editing.
Nurse / Doctor	Assigned encounter, clinical history, screening, documentation and orders.	Access is assignment- and role-scoped.
Technician / Pharmacist	Assigned service execution or medicine event.	Cannot create the originating clinical decision.
Cashier	Invoice, payment, refund and payment status.	Cannot modify clinical records.
Manager / Administrator / Auditor	Reports, configuration or audit review.	Each role has a separate scope.
Patient (Phase 2)	Appointment request and released-document viewing.	Verified self-service only.

Table 4: Role-to-activity mapping and interaction boundaries

2.3 Business Workflow and Controls

Figure 1 shows the normal outpatient journey. Figure 2 isolates appointment and arrival exceptions, where invalid transitions are most likely to disrupt service flow.
The workflow is governed by five control points:

1. Search and duplicate checks occur before a new patient record is created.
2. Booking and rescheduling validate provider, room and required-resource availability before commitment.
3. Check-in creates one valid active Encounter and its queue state.
4. Orders, execution and medicine events are recorded as separate lifecycle events.
5. Encounter closure, payment and refund preserve their own accountable history.

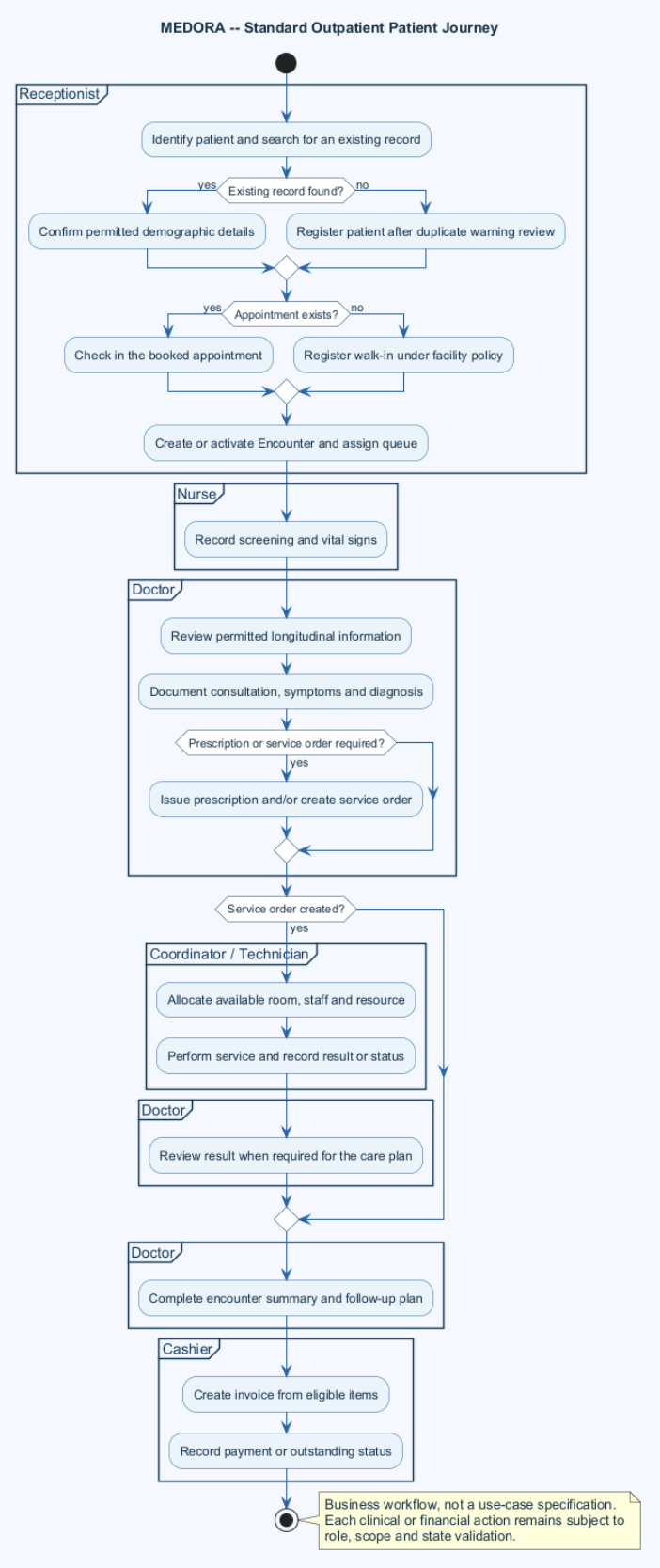


Figure 1: Target business workflow for a standard outpatient journey

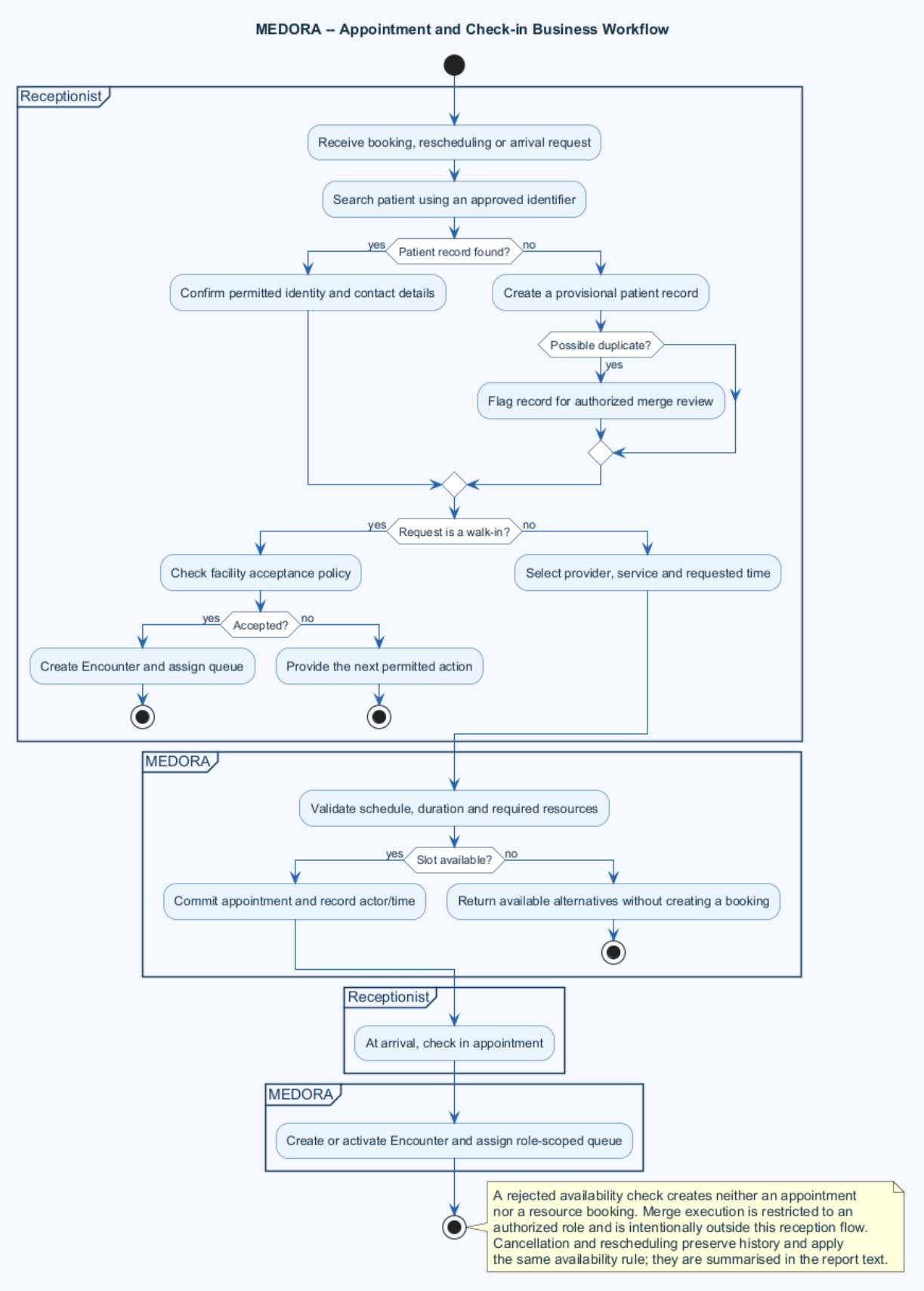


Figure 2: Appointment and check-in workflow with alternative and exception paths

Process	Completion state	Alternative or exception
Identity and booking	Confirmed patient and valid appointment.	Duplicate review or an alternative slot; no conflict is committed.
Arrival and queue	One active encounter with queue status.	Accepted walk-in or rejected duplicate active encounter.
Consultation and documentation	Structured clinical note and plan.	Draft is retained; signed content is amended, not overwritten.
Order and execution	Order, allocation and recorded result.	Unavailable resource, delay or cancellation preserves status history.
Closure and finance	Encounter summary plus invoice/payment status.	Incomplete work blocks closure; refund retains reason and history.

Table 5: Major business processes, outcomes and controlled alternatives

2.4 Business Rules

ID	Rule	Required behaviour
BR-001	Patient identity	Use a stable identifier, flag possible duplicates and preserve merge history.
BR-002	Availability	Validate provider, room, staff and required resources before booking or allocation.
BR-003	Encounter integrity	Prevent duplicate active encounters and maintain role-scoped queues.
BR-004	Authorisation and audit	Enforce role, scope and state checks at the server boundary; retain sensitive-event evidence.
BR-005	Clinical-record integrity	Preserve signed notes; record corrections as linked amendments with a reason.
BR-006	Clinical safety and execution	Warnings inform clinicians without deciding treatment; prescriptions, orders and actual execution remain distinct.
BR-007	Financial history	Keep invoices, payments and refunds separate from clinical records and retain transaction history.

Table 6: Core business rules

2.5 Domain Model and Core Entities

Figure 3 presents the clinical core. The remaining operational and governance concepts extend this core without changing its central relationship: the Encounter contextualises the clinical work performed for one Patient.

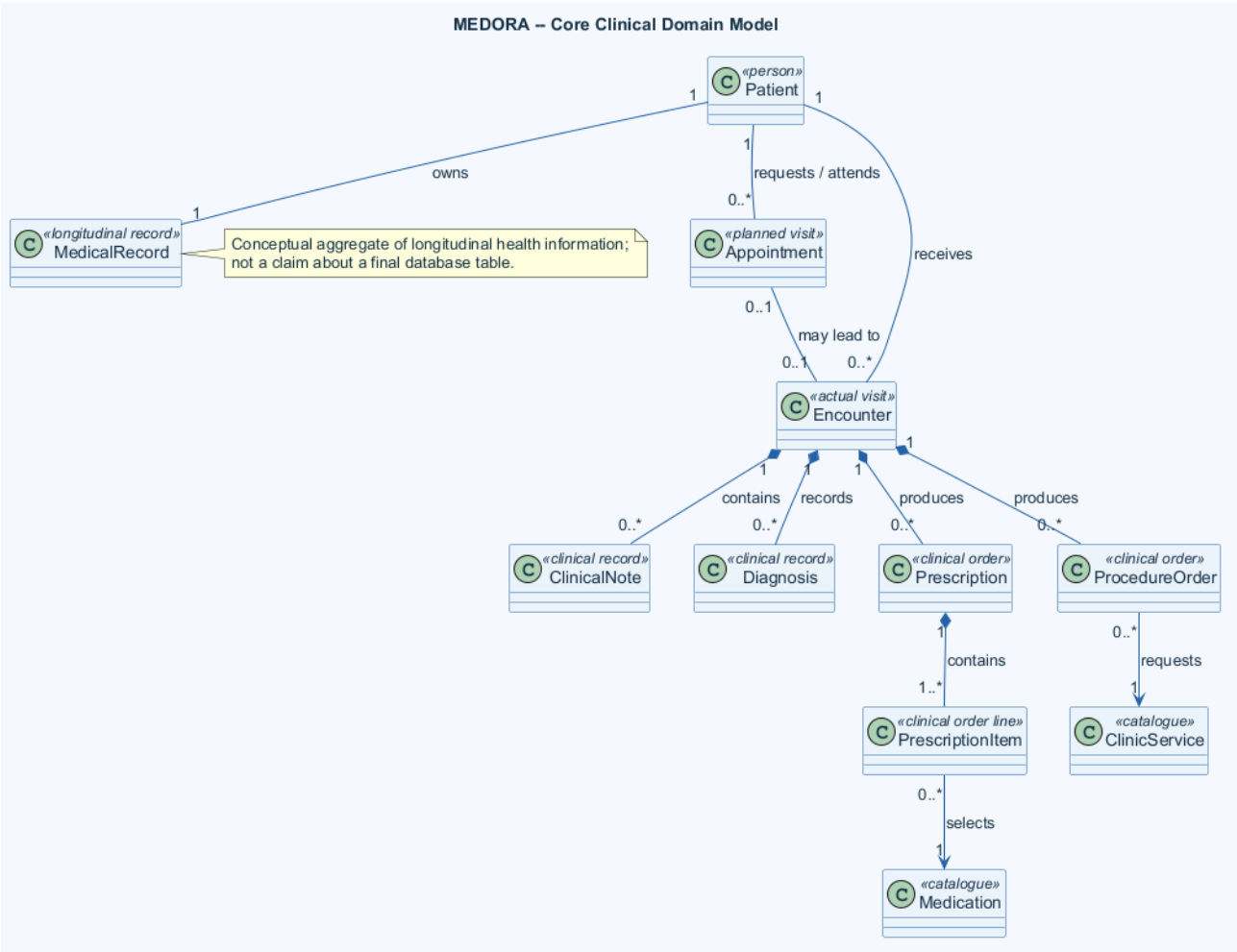


Figure 3: Core clinical conceptual domain model

Entity group	Purpose
Patient and MedicalRecord	Stable identity and longitudinal clinical context.
Appointment and Encounter	Planned visit versus actual outpatient episode.
ClinicalNote, Diagnosis and Prescription	Authorised clinical documentation and decisions.
ProcedureOrder and ResourceBooking	Requested service, constrained allocation and execution context.
Invoice and Payment	Charges and financial transactions without clinical mutation.
UserAccount, Role, Attachment and AuditLog	Scoped access, controlled documents and accountability evidence.

Table 7: Core entity glossary

Relationship	Business meaning
Patient–Appointment–Encounter	A patient may have many appointments and encounters; an appointment may lead to one encounter, while a walk-in has none.
Encounter–clinical records	Notes, diagnoses, prescriptions and procedure orders belong to the encounter in which they are created.
Prescription–medicine event	A prescription contains medicine directions; dispensing or administration is a separate, accountable event.
ProcedureOrder–ResourceBooking–Result	An order may reserve resources and later obtain an execution result; neither is implied by the other.
Invoice–Payment	One invoice may have zero or many payment, refund or adjustment transactions.
UserAccount–Role–AuditLog	Role and scope govern access; audit evidence records the actor, action, target and time.

Table 8: Core domain relationships

Business need	System implication	Trace
Reliable identity	Search, duplicate control, merge history and longitudinal record.	BR-001; FR-003–FR-004
Conflict-free coordination	Transactional availability, queue state and resource allocation.	BR-002–BR-003; FR-005–FR-006, FR-015
Accountable care	Signed-note amendments, warnings and separate execution events.	BR-005–BR-008; FR-009–FR-018
Financial accountability	Invoice, payment and refund history separate from clinical data.	BR-007; FR-022–FR-023
Governance	Least privilege, audit trail, reports and controlled export.	BR-004; FR-026–FR-030

Table 9: Business-to-system traceability

3 System Requirements Analysis

3.1 Functional Decomposition and Package Structure

Figure 4 groups MEDORA by business capability rather than user interface. It provides the functional boundary for the requirements and use-case model.
Identity and Access provides the stable patient and user context on which all other capabilities rely. Scheduling and Reception controls planned appointments, walk-ins, queues and resource availability. Clinical Care records the actual encounter; Orders and Execution preserves the distinction between a decision and the action carried out. Finance and Operations handles eligible charges and operational visibility, while Governance applies cross-cutting controls for attachments, audit evidence and exports. The package dependencies are deliberately not shown in Figure 4: they would duplicate the logical architecture and reduce readability.

3.2 Actors and Overall Use Case Model

The overall use-case model in Figure 5 gives a bounded view of actor goals. Patient self-service is explicitly Phase 2; in the MVP, reception staff perform appointment and check-in work on the patient's behalf. No insurer, payment gateway, laboratory information system, imaging system or messaging provider is shown as an actor because no approved integration contract exists.

3.3 Use Case Diagrams by Functional Group

The focused diagrams below decompose the overview without changing the actor boundary. Solid associations identify participating external roles. A dashed `<<include>>` relationship is mandatory reusable behaviour; `<<extend>>` is conditional behaviour that leaves the base use case valid. UC-054 (operational notification) and UC-055 (released-document viewing) are explicitly scoped as P1/Phase 2 capabilities and are not implementation claims. The sources and mappings are maintained in the model inventory under `doc/asset/03-system-requirements/use-case-model`.

3.3.1 Patient Management

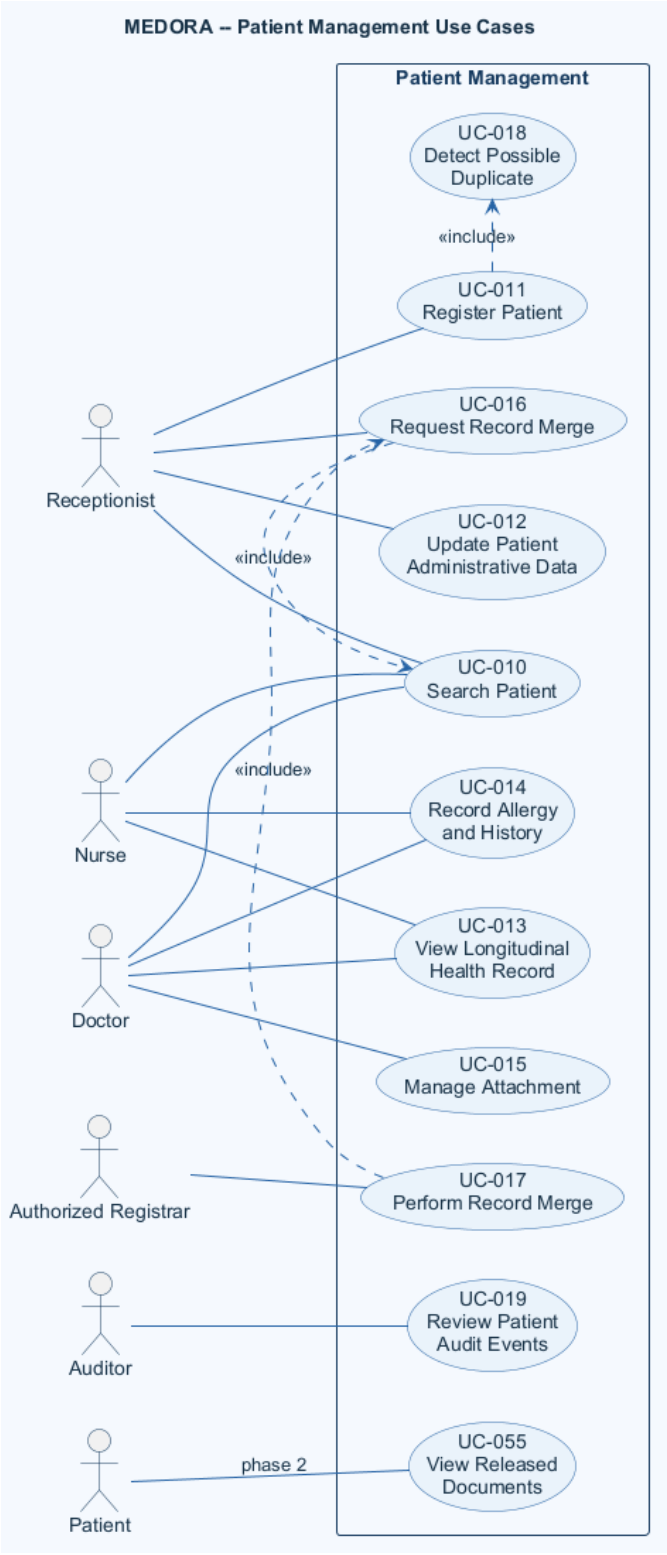


Figure 6: Patient-management use cases

3.3.2 Appointment, Check-in and Queue Management

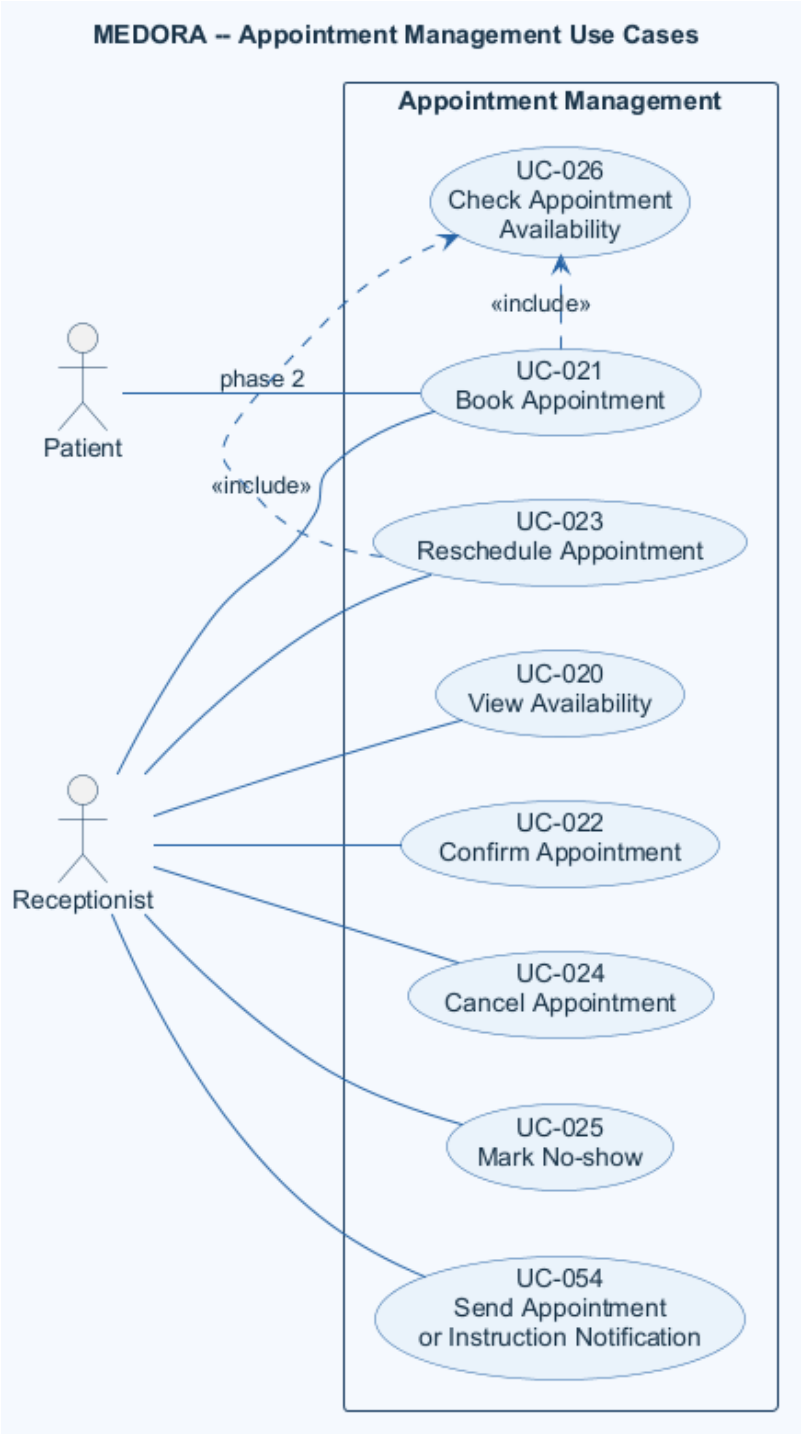


Figure 7: Appointment-management use cases

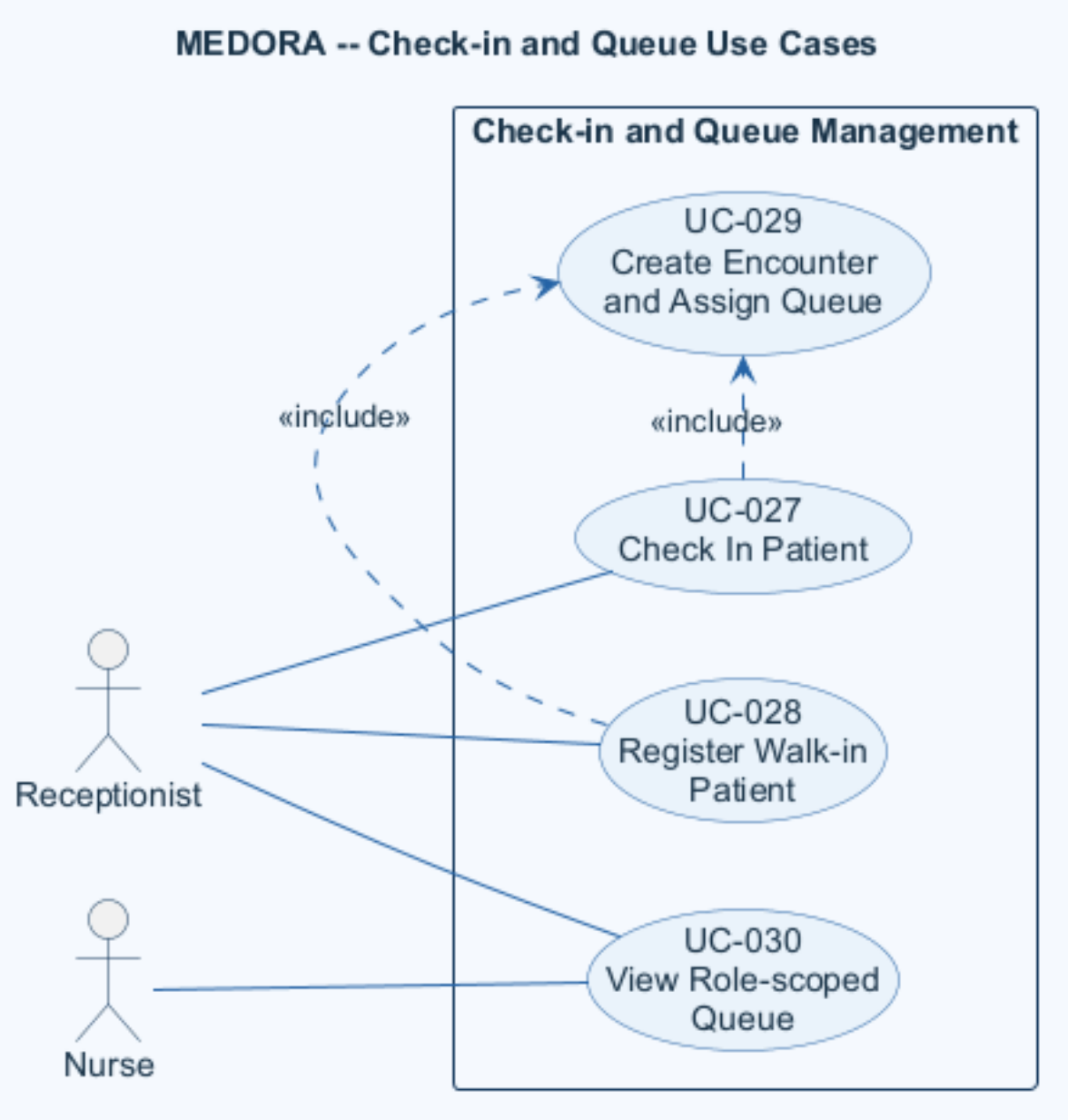


Figure 8: Check-in and queue-management use cases

3.3.3 Medical Examination and Encounter Management

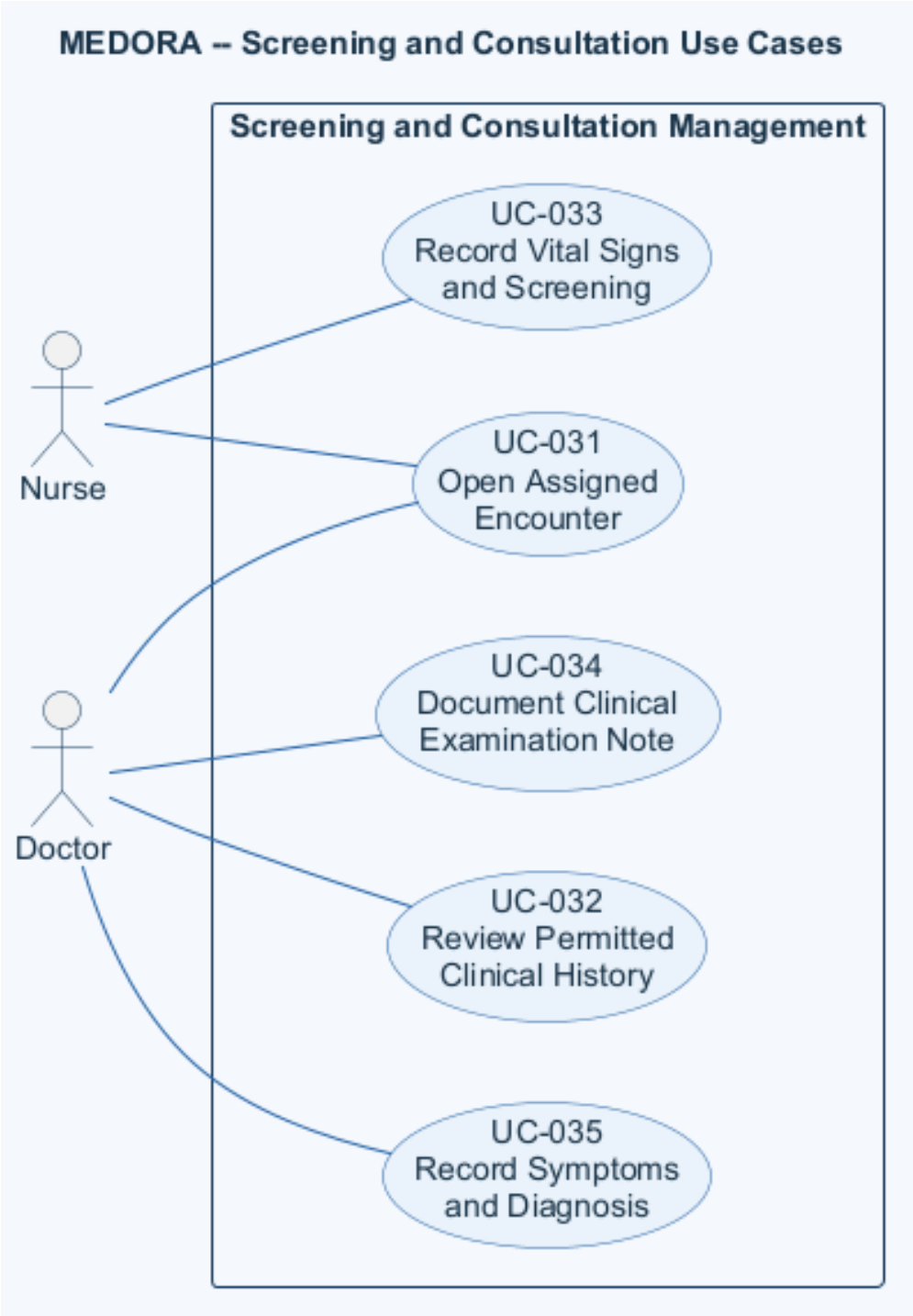


Figure 9: Screening and consultation use cases

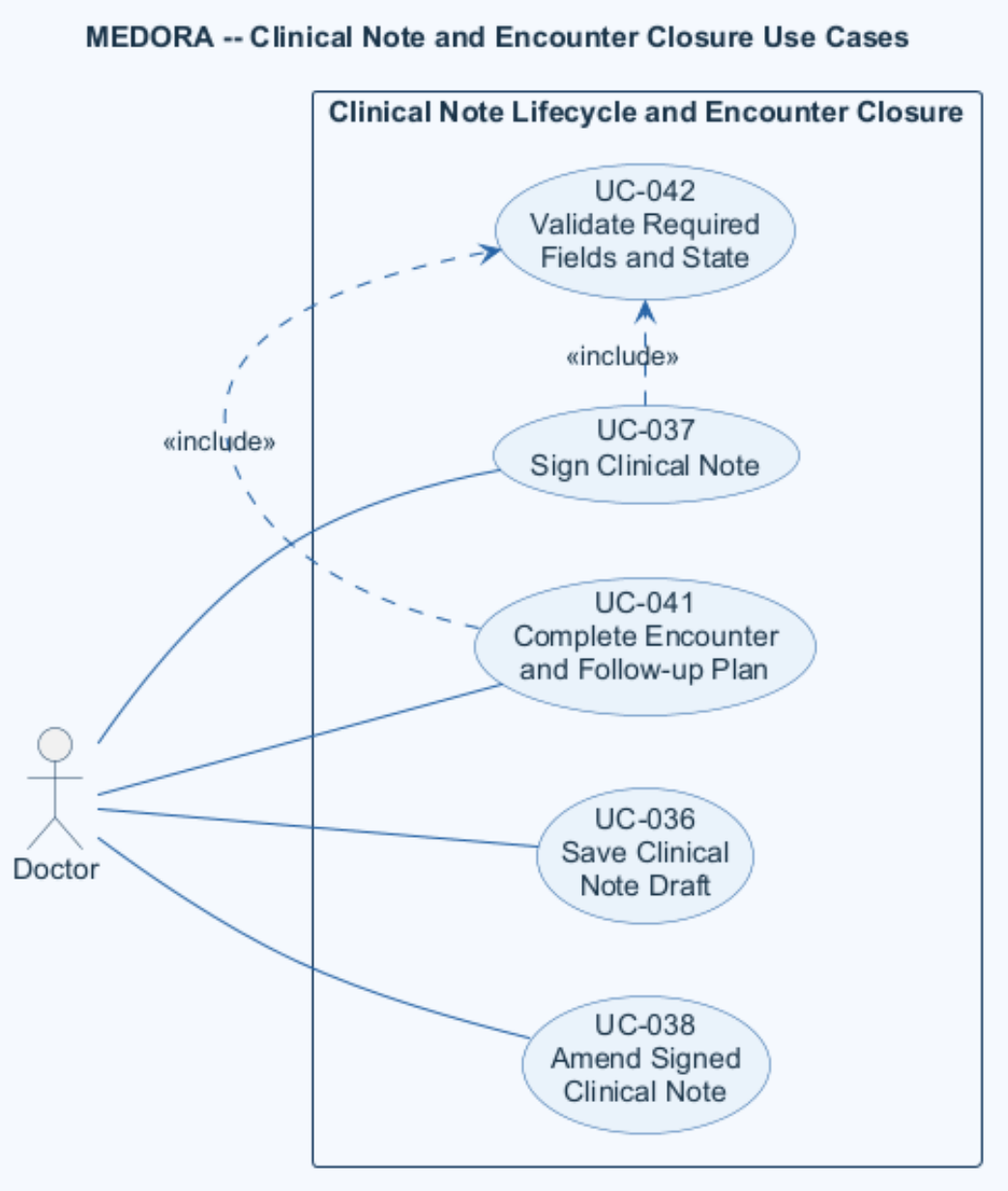


Figure 10: Clinical-note lifecycle and encounter-closure use cases

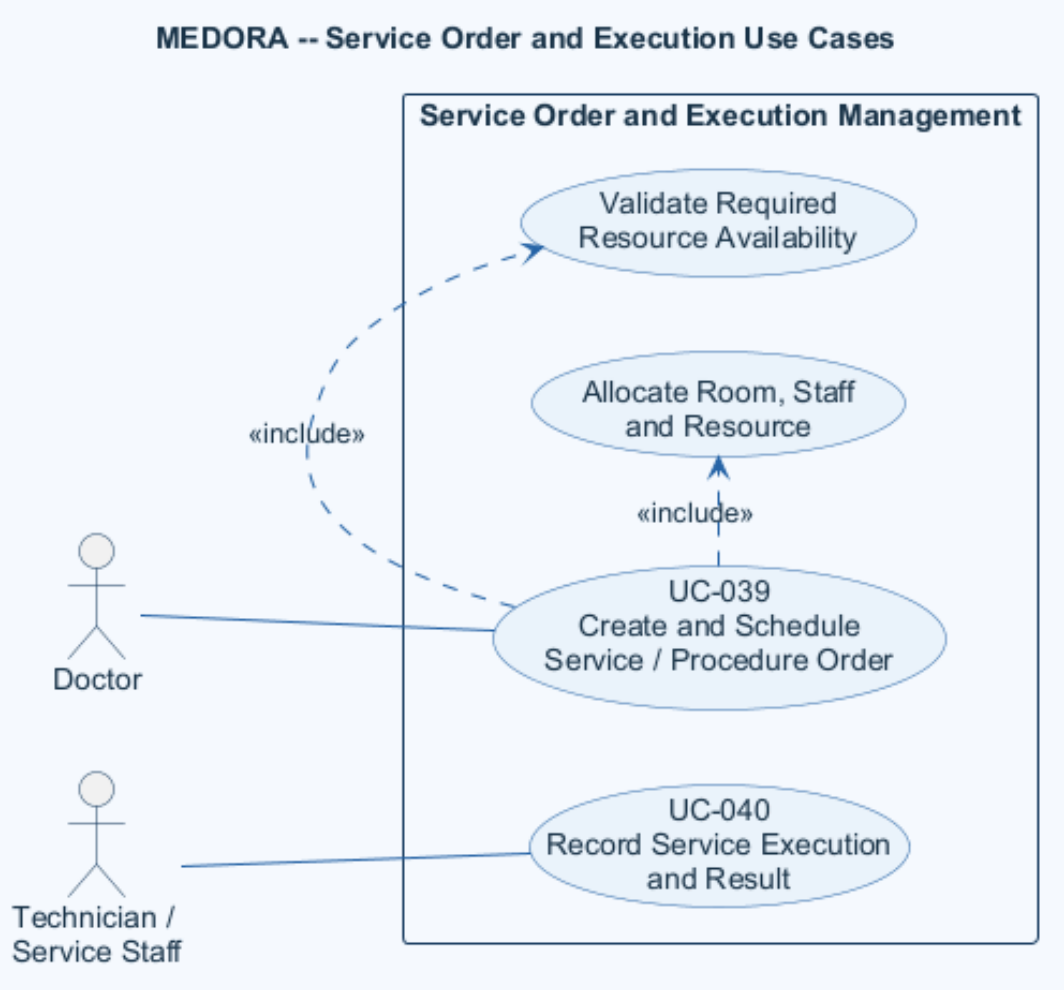


Figure 11: Service-order and execution use cases

3.3.4 Prescription, Billing and Governance

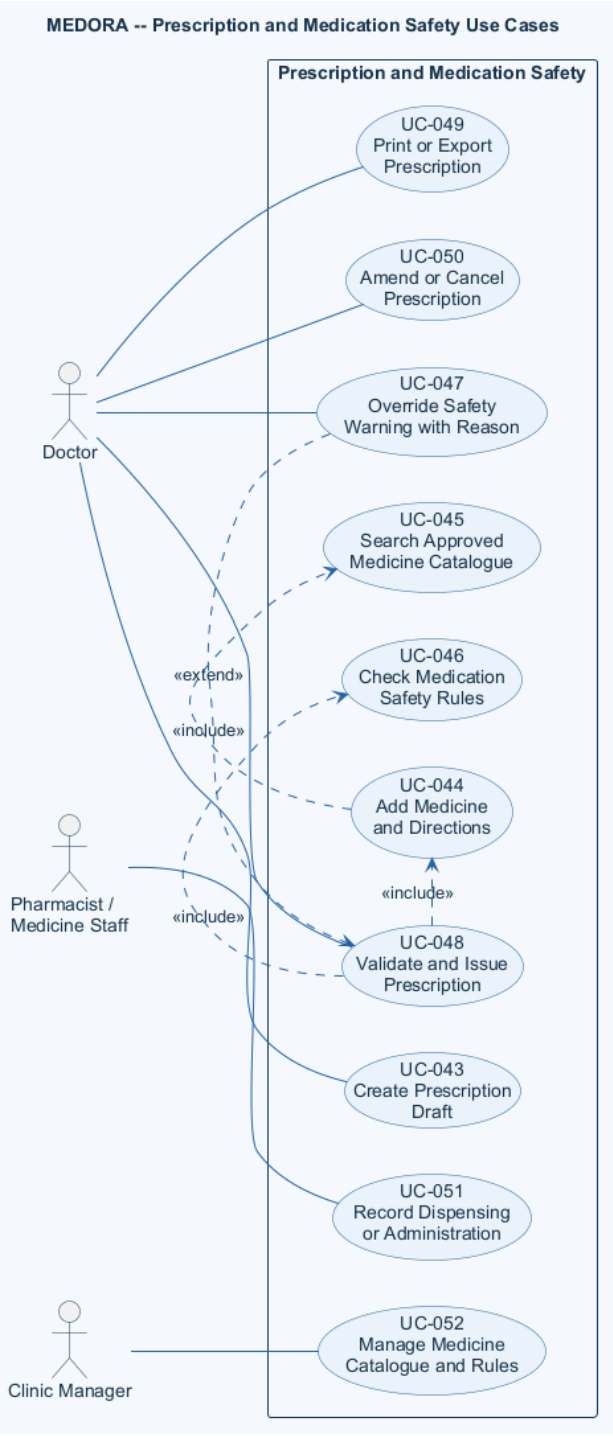


Figure 12: Prescription and medication-safety use cases

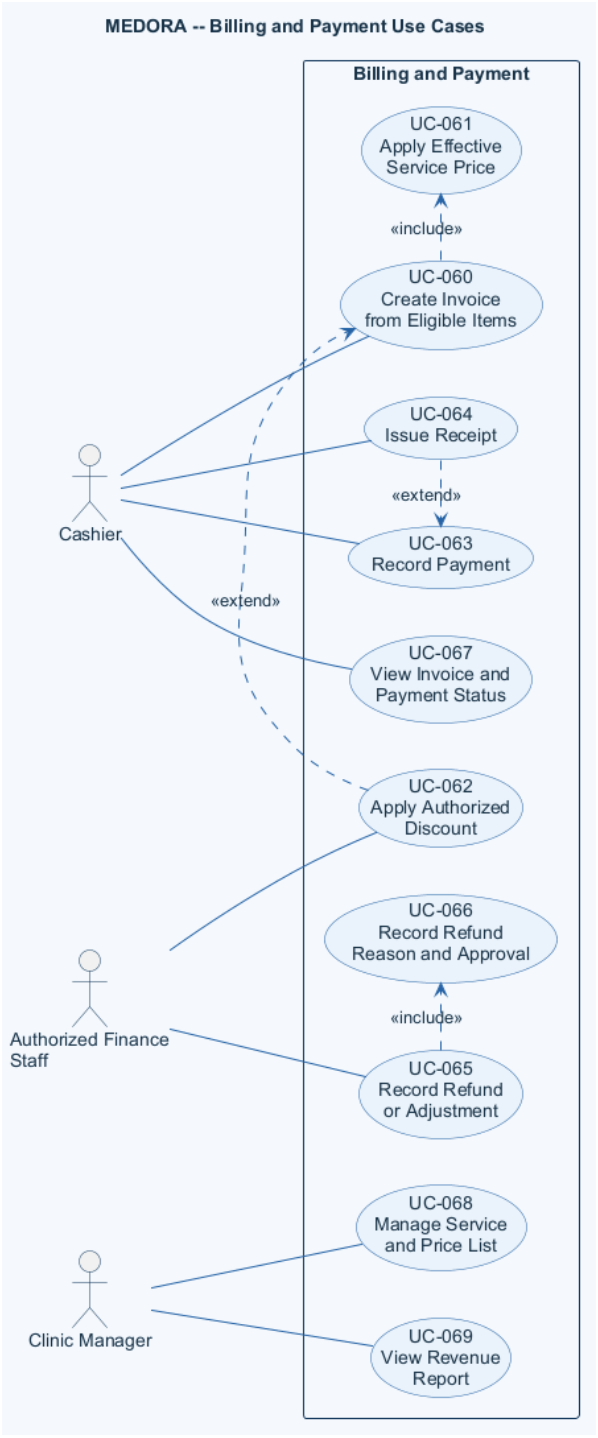


Figure 13: Billing and payment use cases

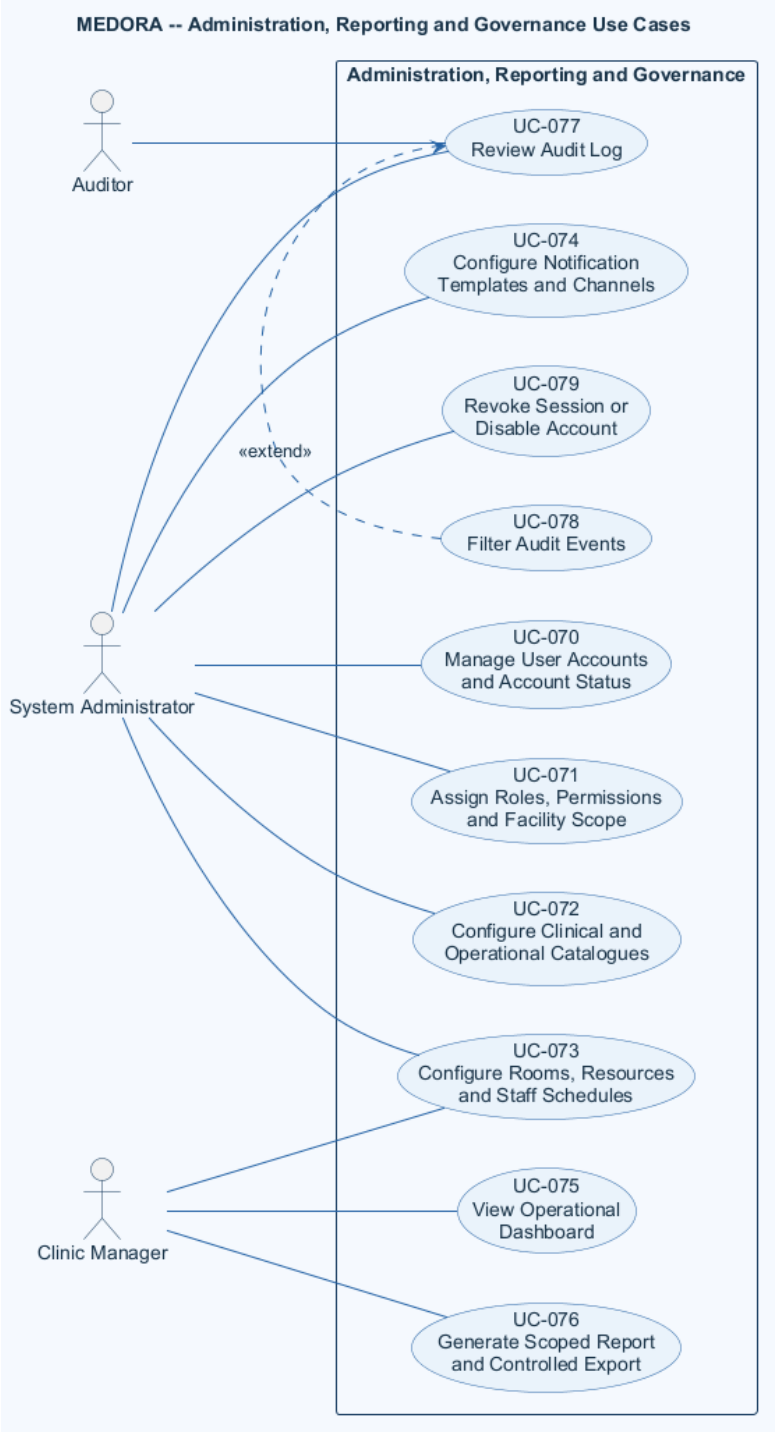


Figure 14: Administration, reporting and governance use cases

3.4 Detailed Use Case Specifications

The six use cases below carry the principal state, safety or financial risk. Each specification states the minimum successful path and the control that prevents an invalid state transition.

3.4.1 UC-021 Book Appointment

Item	Specification

Actor, trigger and preconditions	Receptionist; a patient and service/provider context are selected; the actor is authorised.
Success flow	Select slot; validate provider, room, duration and required resources; atomically create the Appointment and audit-relevant metadata.
Alternative and exception flow	Rescheduling repeats validation and preserves history. An unavailable slot, unresolved identity or invalid scope creates no appointment.
Postconditions and data	One valid Appointment exists; no Encounter is created by booking alone.
Rules and trace	BR-002, BR-004; FR-005, FR-017; UC-026; NFR-06, NFR-15.

3.4.2 UC-027 Check In Patient

Item	Specification
Actor, trigger and preconditions	Receptionist receives an identified booked patient or an accepted walk-in.
Success flow	Confirm arrival; check the active-encounter rule; create/activate the Encounter; assign the role-scoped queue state.
Alternative and exception flow	An accepted walk-in follows the same queue rule. A conflicting active encounter, rejected walk-in or invalid scope creates no duplicate encounter.
Postconditions and data	The relevant Appointment is updated and one active Encounter with queue status is available to authorised roles.
Rules and trace	BR-003, BR-004; FR-006; UC-029; NFR-06, NFR-15.

3.4.3 UC-037/UC-038 Sign and Amend a Clinical Note

Item	Specification
Actor, trigger and preconditions	Authorised Doctor; a draft note belongs to an assigned Encounter.
Success flow	Validate required content and state; sign the note with actor/time. A correction creates a linked amendment with a reason.
Alternative and exception flow	A draft may be saved for later. Missing fields, invalid state, invalid scope or a concurrent update prevents signing or requires reconciliation.
Postconditions and data	Signed content remains intact; each amendment retains author, time, reason and linkage.
Rules and trace	BR-004, BR-005; FR-009, FR-020, FR-032; UC-042; NFR-06, NFR-07, NFR-15.

3.4.4 UC-048 Validate and Issue Prescription

Item	Specification
Actor, trigger and preconditions	Authorised Doctor requests issue for a valid encounter with complete medicine items.
Success flow	Validate dose and directions; evaluate configured safety rules; address a warning or record a permitted override; issue the prescription.
Alternative and exception flow	The Doctor may retain a draft. Missing fields, an unresolved required warning or invalid scope blocks issue; unavailable source data does not create an invented warning.
Postconditions and data	An issued prescription and item history exist; no dispensing or administration event is inferred.
Rules and trace	BR-006, BR-008; FR-011–FR-013; UC-046–UC-047; NFR-06, NFR-07.

3.4.5 UC-039/UC-040 Service Order, Allocation and Result

Item	Specification
Actor, trigger and preconditions	Doctor creates an order for an active encounter; assigned Technician/Service Staff records execution.
Success flow	Record service, reason and priority; validate/allocate required room, staff and resource; record performer, time, status and result.
Alternative and exception flow	An order may be scheduled later or remain pending. Unavailable resources, unassigned staff, delay or cancellation preserve a valid non-completed state.
Postconditions and data	ProcedureOrder, optional ResourceBooking and execution/result history are linked to the Encounter.
Rules and trace	BR-002, BR-004, BR-007; FR-014–FR-018; NFR-06, NFR-15.

3.4.6 UC-063/UC-065 Record Payment and Refund

Item	Specification
Actor, trigger and preconditions	Cashier records payment for an eligible Invoice; authorised Finance Staff records a refund or adjustment.
Success flow	Verify outstanding status; record method, amount and time; update the Invoice. Refunds record the prior transaction, reason and required approval.
Alternative and exception flow	Partial payment leaves an outstanding balance. Invalid amount/status, invalid scope or duplicate submission produces no financial effect.
Postconditions and data	Payment/refund history and invoice status are updated; clinical content is unchanged.

Rules and trace	BR-004, BR-007; FR-022–FR-023; UC-064, UC-066; NFR-06, NFR-07, NFR-15.

3.5 Functional Requirements

Table 16 groups related requirements without changing their identifiers. P0 is required for the MVP; P1 is a controlled extension; P2 depends on a later organisational decision or integration contract.

Table 16: Functional requirements catalogue

ID	Capability	Observable system behaviour	Priority
FR-001–FR-002	Authentication and access	Authenticate individual accounts; manage roles, facility scope and sensitive-session controls; enforce authorisation at the server boundary.	P0
FR-003–FR-004	Patient identity	Create/search patient profiles, flag duplicates and preserve authorised merge evidence.	P0/P1
FR-005–FR-006	Appointment and reception	Manage appointment status, availability, walk-ins, check-in and role-scoped queues.	P0
FR-007–FR-010	Encounter documentation	Record screening, history, clinical notes, symptoms, diagnoses and follow-up in encounter context.	P0
FR-011–FR-013	Prescription and medicine event	Validate configured safety rules; issue prescriptions; record dispensing/administration separately.	P0/P1
FR-014–FR-018	Service order and execution	Create orders; allocate rooms, staff and resources; record execution status and result.	P0
FR-019	Diagnostic request	Support an approved manual request/result attachment or later adapter.	P1
FR-020–FR-021	Encounter conclusion	Record conclusion and follow-up; support configured leave/discharge review where applicable.	P0/P1
FR-022–FR-023	Catalogue, invoice and payment	Maintain effective service/price data; create invoices and retain payment/refund history.	P0
FR-024–FR-025	Notification and patient portal	Send configured operational notifications; support verified portal self-service only when enabled.	P1 / Phase 2
FR-026–FR-027	Dashboard, reports and export	Present scoped operational indicators and log controlled report exports.	P0
FR-028–FR-031	Administration, attachment and search	Maintain reference catalogues, controlled attachments, audit evidence and scoped search.	P0
FR-032	Draft and recovery	Preserve drafts and unsaved-change state; report success only after server confirmation.	P1
FR-033–FR-034	Facility and external integration	Apply approved facility scope and provide logged connectors only for approved contracts.	P2

3.6 Non-functional Requirements

The table defines the quality targets used for design and later verification.

Table 17: Non-functional requirements catalogue

ID	Attribute	Requirement / acceptance basis	Status
NFR-01–NFR-03	Performance, availability and recovery	Target p95 ordinary screen $\leq$ 2 seconds, search $\leq$ 3 seconds; define availability, RPO and RTO before deployment.	Target

ID	Attribute	Requirement / acceptance basis	Status
NFR-04–NFR-07	Security and auditability	Use protected transport/storage, server-side RBAC, least privilege, time-synchronised sensitive-event evidence and retention policy.	Specified
NFR-08–NFR-09	Resilience and scale	Preserve confirmed data through dependency failure; use bounded queries, pagination and modular APIs.	Specified
NFR-10–NFR-11	Compatibility and accessibility	Support current Chrome/Edge desktop/tablet use with keyboard operation, labels, contrast and clear errors.	Target
NFR-12–NFR-13	Maintainability and observability	Use controlled migrations, documentation and structured operational metrics/logs.	Specified
NFR-14–NFR-15	Locale and integrity	Configure Vietnamese/local formats; use transactions, constraints and idempotency for critical operations.	Specified
NFR-16	Testability	Maintain unit, integration and end-to-end tests with non-production data.	Specified

3.7 Role-based Access Control Matrix

Table 18 records the intended minimum business permissions. It is a requirements baseline, not an implementation claim. **R**, **C**, **U**, **S**, **A** and **F** respectively mean read, create, update, sign/finalise, approve/override and finance transaction. A dash means no routine access. All non-dash permissions remain subject to facility, assignment and lifecycle state; Section 4 must enforce them at the server boundary.

3.8 Requirements Traceability and Use Case Validation

Table 19 links the business rationale to requirements, use cases, entities and planned verification.

Table 19: Requirements traceability and planned verification

Business rule/need	Requirement	Use cases	Domain/design focus	Planned verification
BR-001 identity continuity	FR-003, FR-004, FR-031	UC-010–UC-018	Patient, MedicalRecord, AuditLog; unique identifier and merge history.	Duplicate-search, unauthorised-merge and merge-history test cases.
BR-002 conflict-free allocation	FR-005, FR-015–FR-017; NFR-15	UC-021–UC-026, UC-039	Appointment, ResourceBooking, schedule/room/staff constraints.	Concurrent booking/reschedule tests; zero accepted conflict target.
BR-003 controlled arrival	FR-006	UC-027–UC-030	Appointment–Encounter relationship and queue state.	Duplicate-active-encounter and role-scoped queue tests.
BR-005 signed-record integrity	FR-009, FR-032; NFR-07, NFR-15	UC-036–UC-038, UC-041–UC-042	ClinicalNote draft/signed/amendment lifecycle.	Missing-field, amendment and concurrent-edit tests.

Business rule/need	Requirement	Use cases	Domain/design focus	Planned verification
BR-006 bounded warning	FR-011, FR-012	UC-043–UC-048	Prescription, PrescriptionItem, Medication and override evidence.	Rule-trigger, override-reason and no-autonomous-decision tests.
BR-007 accountable execution	FR-014–FR-018	UC-039–UC-040	ProcedureOrder, ResourceBooking and execution result.	Assignment, unavailable-resource, delay/cancel and result tests.
BR-008 separate medicine event	FR-012, FR-013	UC-048–UC-051	PrescriptionItem versus MedicationAdministration.	Verify issue alone creates no administration event.
BR-009 financial history	FR-022, FR-023; NFR-15	UC-060–UC-069	Invoice, Payment and refund/adjustment history.	Partial-payment, duplicate-submission and refund-reason tests.
BR-004 / BR-010 governance	FR-001, FR-002, FR-027, FR-030; NFR-06, NFR-07	UC-001, UC-070–UC-079	UserAccount, Role, AuditLog and controlled export.	Negative API authorisation and audit-completeness tests.
Operational visibility	FR-026, FR-027	UC-075–UC-076	Scoped measures, report filters and export audit.	Role-scope/report-reconciliation tests.

Open decisions. The clinic will confirm clinical-warning sources, consent and retention policy, refund approval thresholds, branch scope and portal publication rules before deployment.

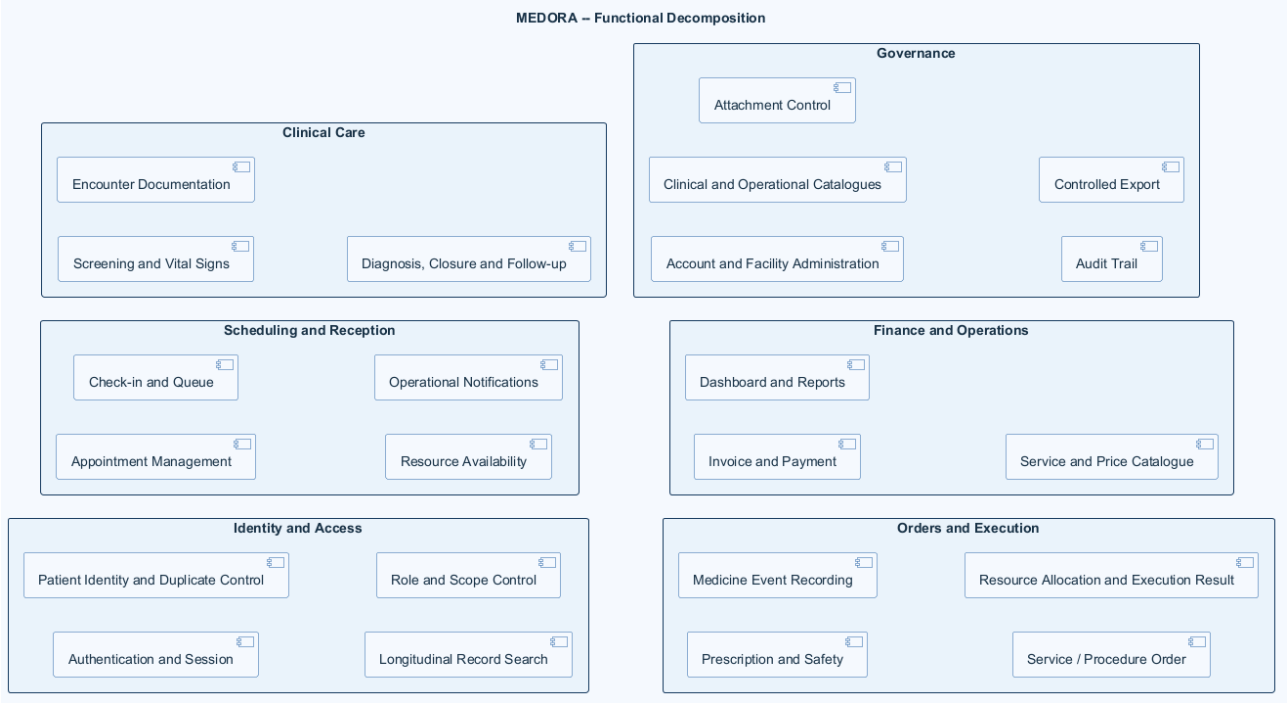


Figure 4: Functional decomposition of MEDORA

Clinical and front-desk roles

Capability / data domain	Rec.	Nurse	Doctor	Tech.	Pharm.
Patient administrative profile	R/C/U	R	R	R†	R†
Appointment, check-in and queue	R/C/U	R	R	R†	–
Screening and vital signs	R	C/U	R	R†	–
Clinical note and diagnosis	R†	R†	R/C/U/S	R†	R†
Prescription and medicine event	R†	R†	R/C/U/S/A	–	R/C/U
Service order and execution result	R†	R/C/U†	R/C/U/S	R/C/U	–
Invoice, payment and refund	R†	–	R†	–	–
Dashboard and scoped reports	–	R†	R†	R†	–
User, role and catalogue configuration	–	–	R†	–	R†
Audit log and controlled export	–	–	R†	–	–

Finance, governance and patient roles

Capability / data domain	Cash.	Manager	Admin	Auditor	Patient*
Patient administrative profile	R†	R†	–	R†	R/U
Appointment, check-in and queue	R†	R	–	R†	R/C/U
Screening and vital signs	–	R†	–	R†	–
Clinical note and diagnosis	–	R†	–	R†	R‡
Prescription and medicine event	–	R†	–	R†	R‡
Service order and execution result	–	R†	–	R†	R‡
Invoice, payment and refund	R/C/U/F	R/A†	–	R†	–
Dashboard and scoped reports	R†	R	–	R†	–
User, role and catalogue configuration	R†	R/U†	R/C/U/A	R†	–
Audit log and controlled export	R†	R†	R†	R	–

Table 18: Target role-based access-control matrix

* Phase 2 only. † Only within assigned, facility or approved scope. ‡ Only information expressly released through the portal. Administrative configuration does not confer clinical-edit authority; Auditor access is read-only and itself may be logged.

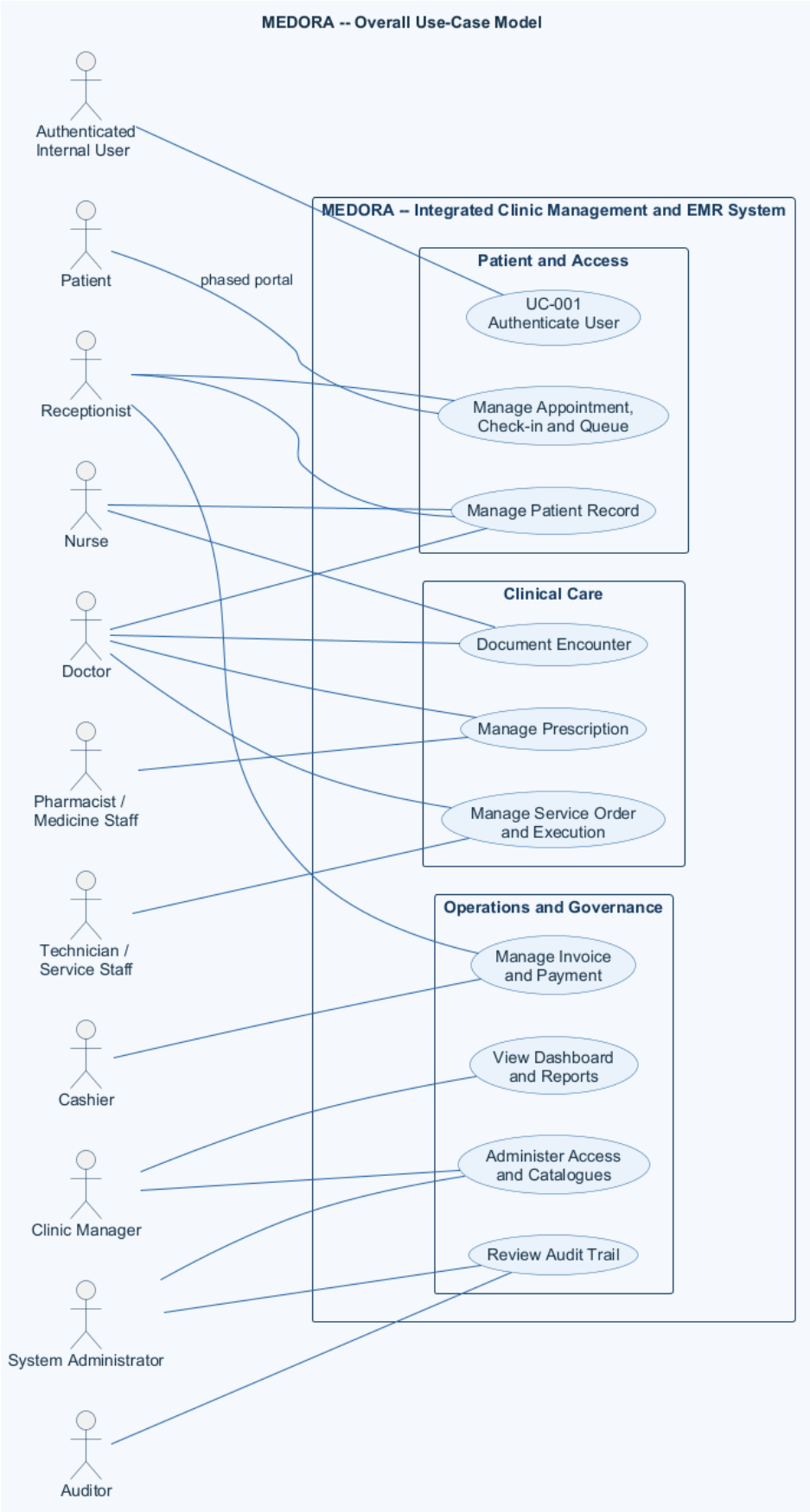


Figure 5: Overall use-case model of MEDORA

4 System Design and Structural Modeling

4.1 System Architecture

4.1.1 Logical Architecture

4.1.2 Deployment Architecture

4.2 Layered Design and Design Patterns

4.2.1 Presentation, Business Logic, and Data Access Layers

4.2.2 API and Web Service Design

4.3 Database Design and Data Integrity

4.3.1 Entity–Relationship Diagram

4.3.2 Relational Schema and Normalization

4.3.3 Persistent Classes and Database Mapping

4.3.4 Keys, Constraints, Data Types, and Multiplicities

4.4 Structural UML Diagrams

4.4.1 Overall Class Diagram

4.4.2 Class Diagrams by Use Case Group

4.4.3 Package Diagram

4.4.4 Component Diagram

4.4.5 Deployment Diagram

4.4.6 Object Diagram (if required)

4.5 Use Case-oriented Design

4.5.1 Participating Classes and Their Responsibilities

4.5.2 Collaboration Between System Components

5 Behavioral Modeling and System Implementation

5.1 Behavioral UML Diagrams

The behavioral models in this section refine the business rules and use cases in Sections 2 and 3. They describe proposed observable system behavior, not calls observed in an implemented application. The six system-sequence diagrams use MEDORA as a black-box boundary so that no unapproved

controller, repository, API, or persistence technology is implied. The activity and state models focus on exception handling, lifecycle integrity, and separation of clinical and financial records.

5.1.1 Sequence Diagrams for Key Use Cases

The sequence views cover appointment booking (UC-021), patient check-in (UC-027), signing and amending a clinical note (UC-037/UC-038), prescription issue (UC-048), service scheduling and execution (UC-039/UC-040), and payment/refund (UC-063/UC-065). Rejected operations leave no partial business effect; successful transitions record the relevant accountable event. The request key in the payment view is a proposed idempotency control derived from NFR-15, not an existing API contract.

5.1.2 Activity Diagrams for Complex Workflows

Figure 21 shows the service lifecycle from clinician order through resource validation and assigned execution. A conflict leaves the order pending without reserving resources; incomplete results do not mark execution complete. Figure 22 makes the boundary between signed clinical completion and invoice settlement explicit: an outstanding balance does not alter the clinical record. These diagrams complement the outpatient and appointment exception workflows in Figures 1 and 2.

5.1.3 State Machine Diagrams for Lifecycle-driven Entities

The state machines formalize the status proposals in WorkStation Sections 5 and 9 and the domain distinctions in Section 2. They are design constraints for later implementation and policy review. In particular, rescheduling preserves the prior appointment history, a clinical-note correction creates a linked version, and refunds remain separate financial transactions.

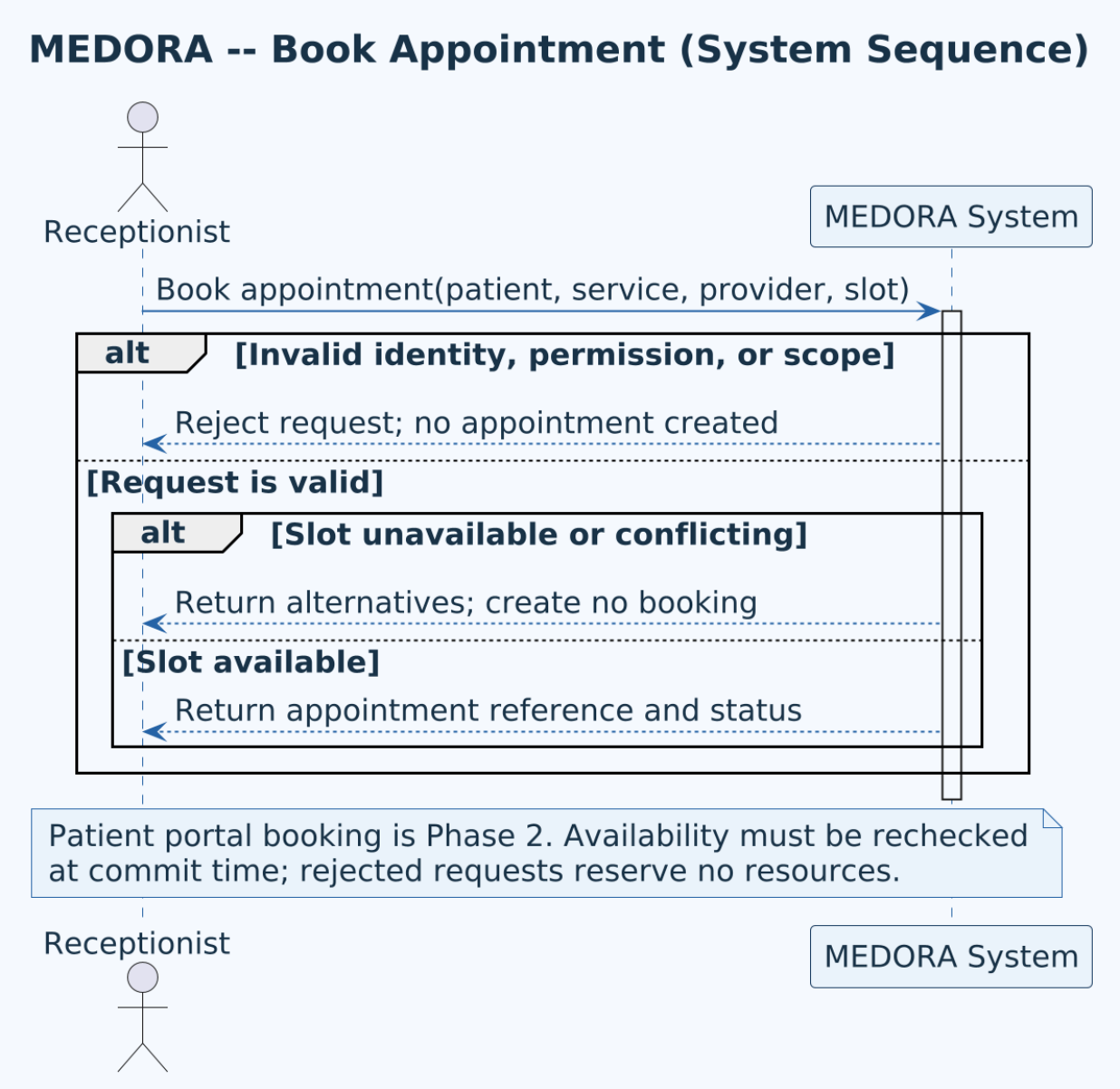


Figure 15: System sequence for appointment booking and availability validation

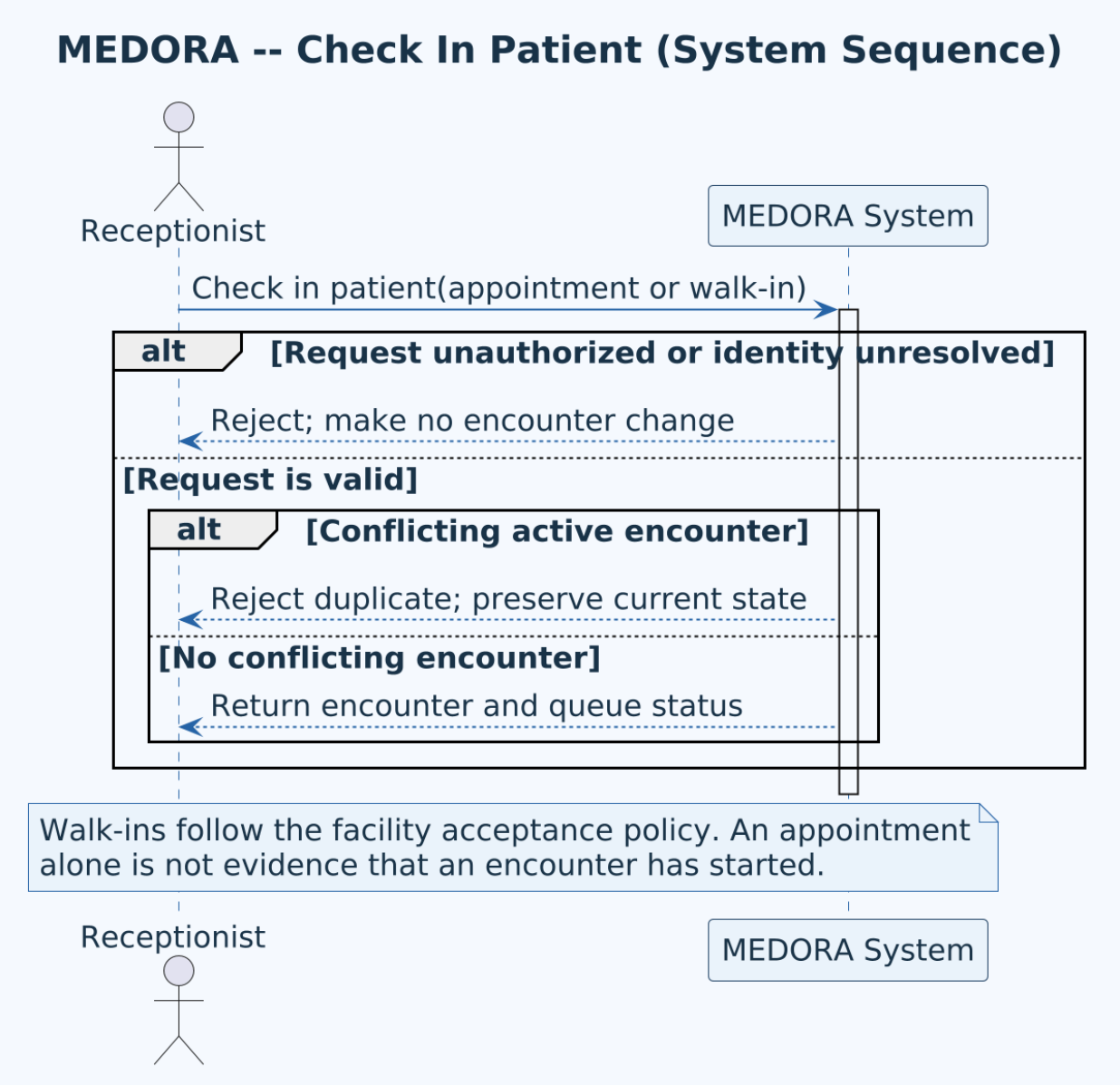


Figure 16: System sequence for check-in, encounter creation, and queue assignment

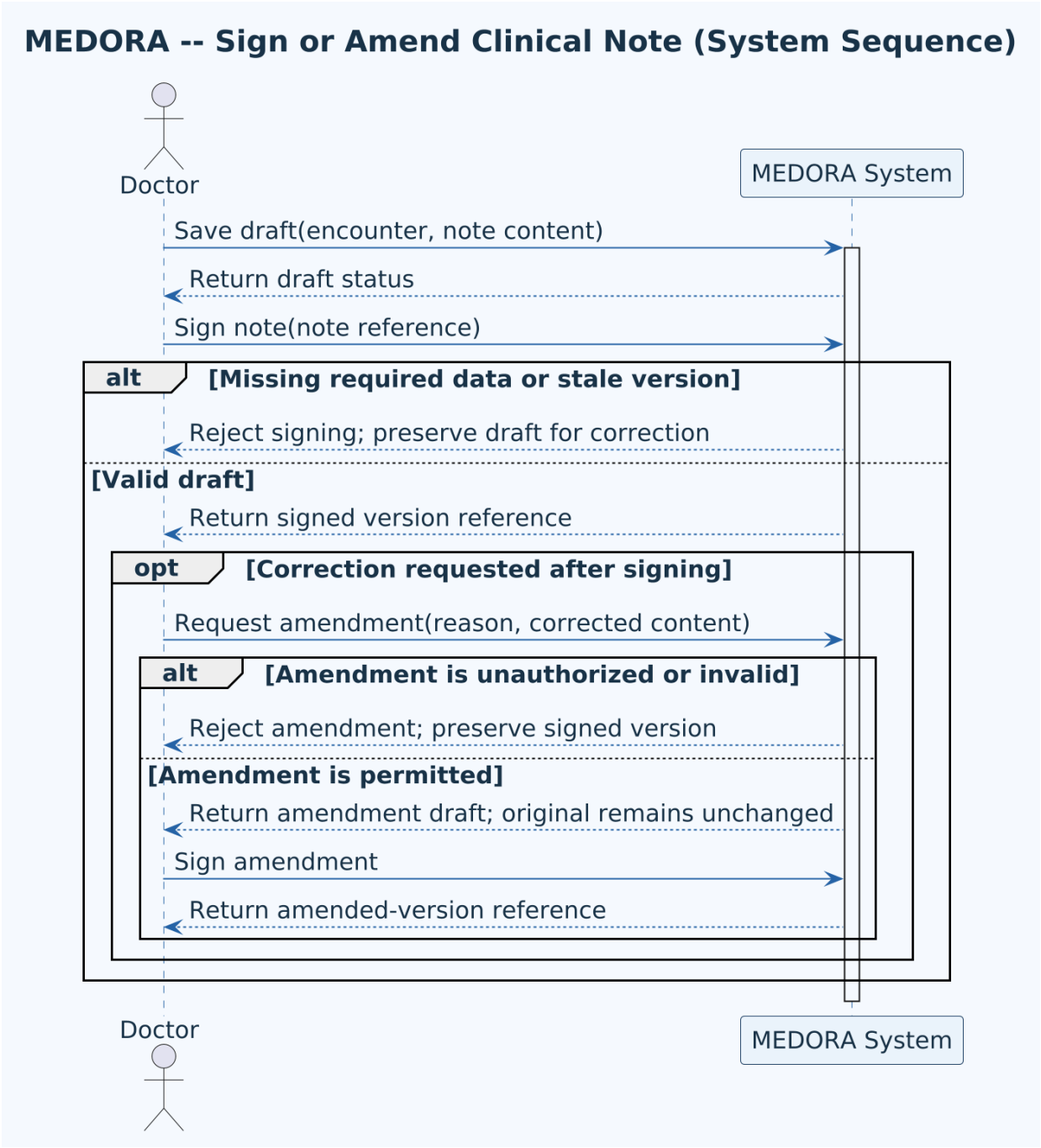


Figure 17: System sequence for signing a clinical note and appending a correction

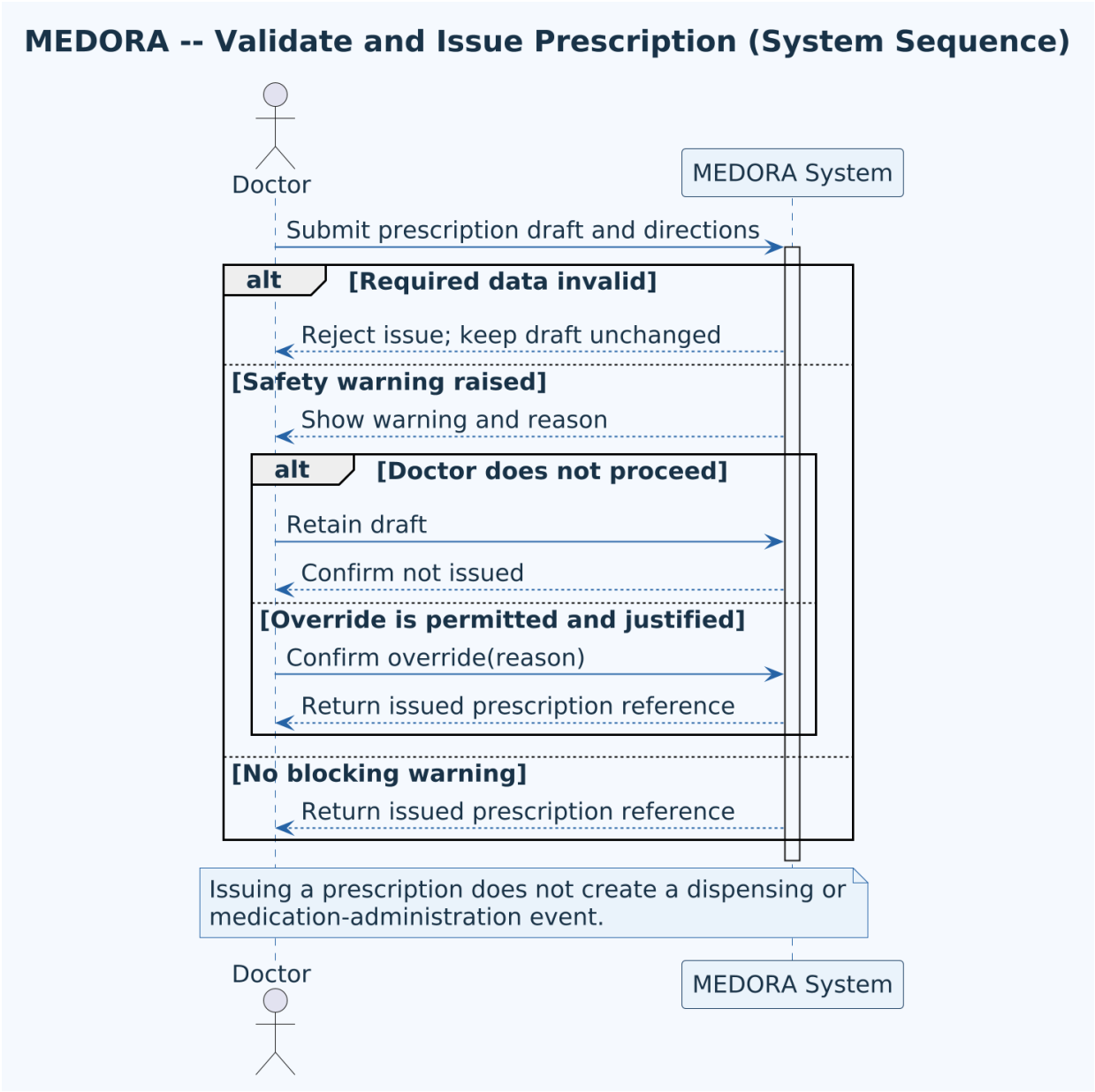


Figure 18: System sequence for configured prescription safety checks and issue

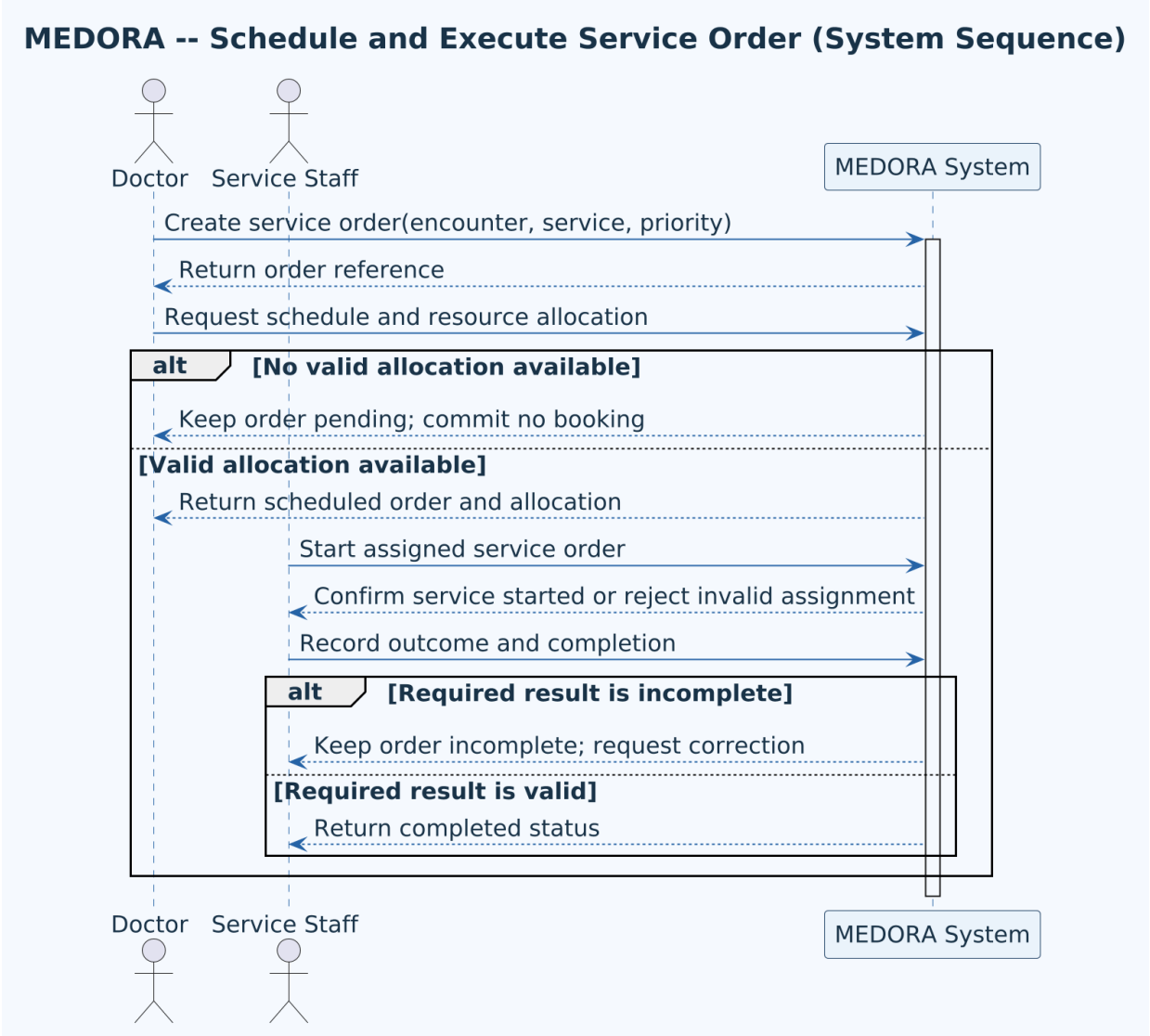


Figure 19: System sequence for resource allocation and service-order execution

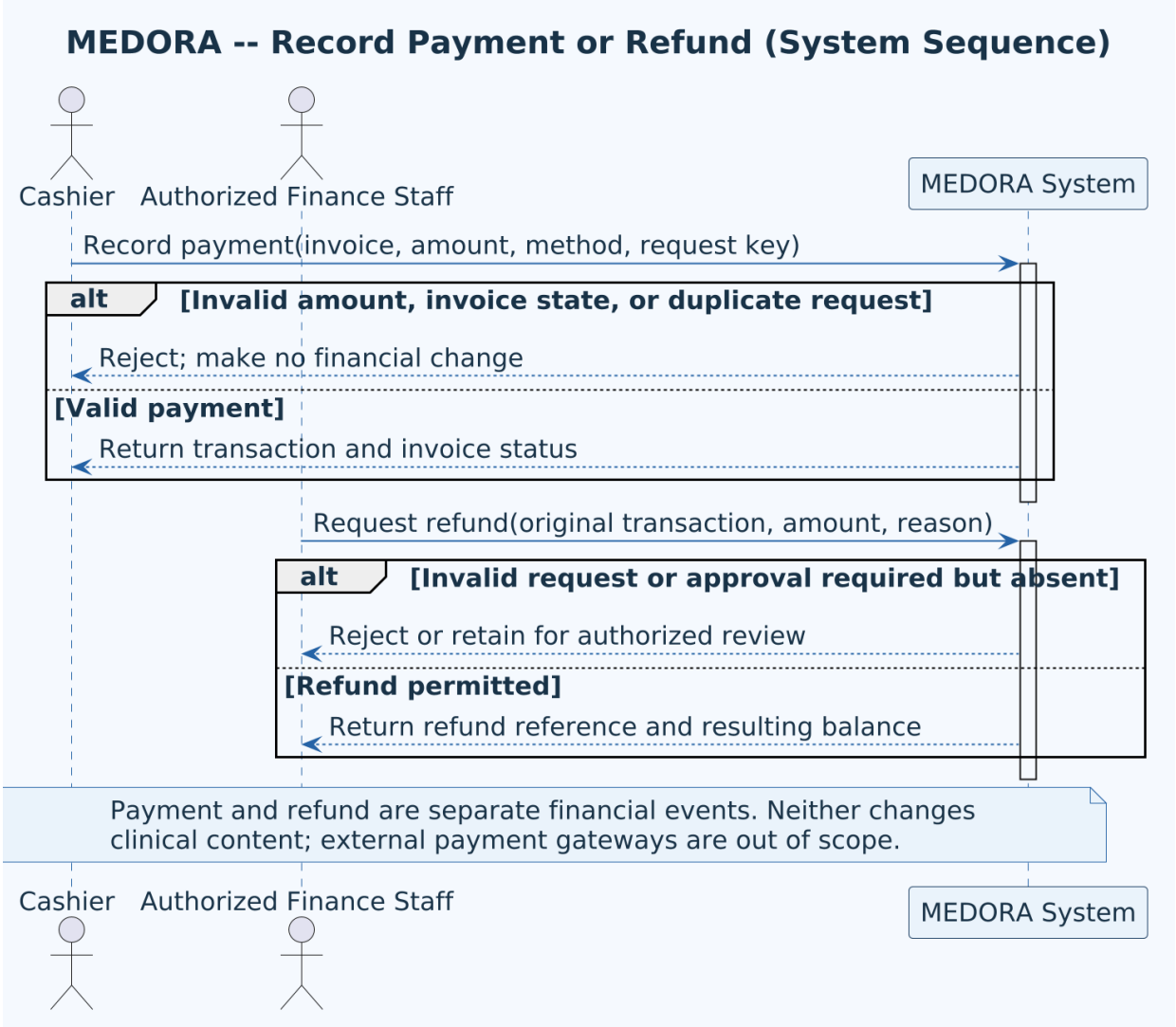


Figure 20: System sequence for payment and separately authorized refund events

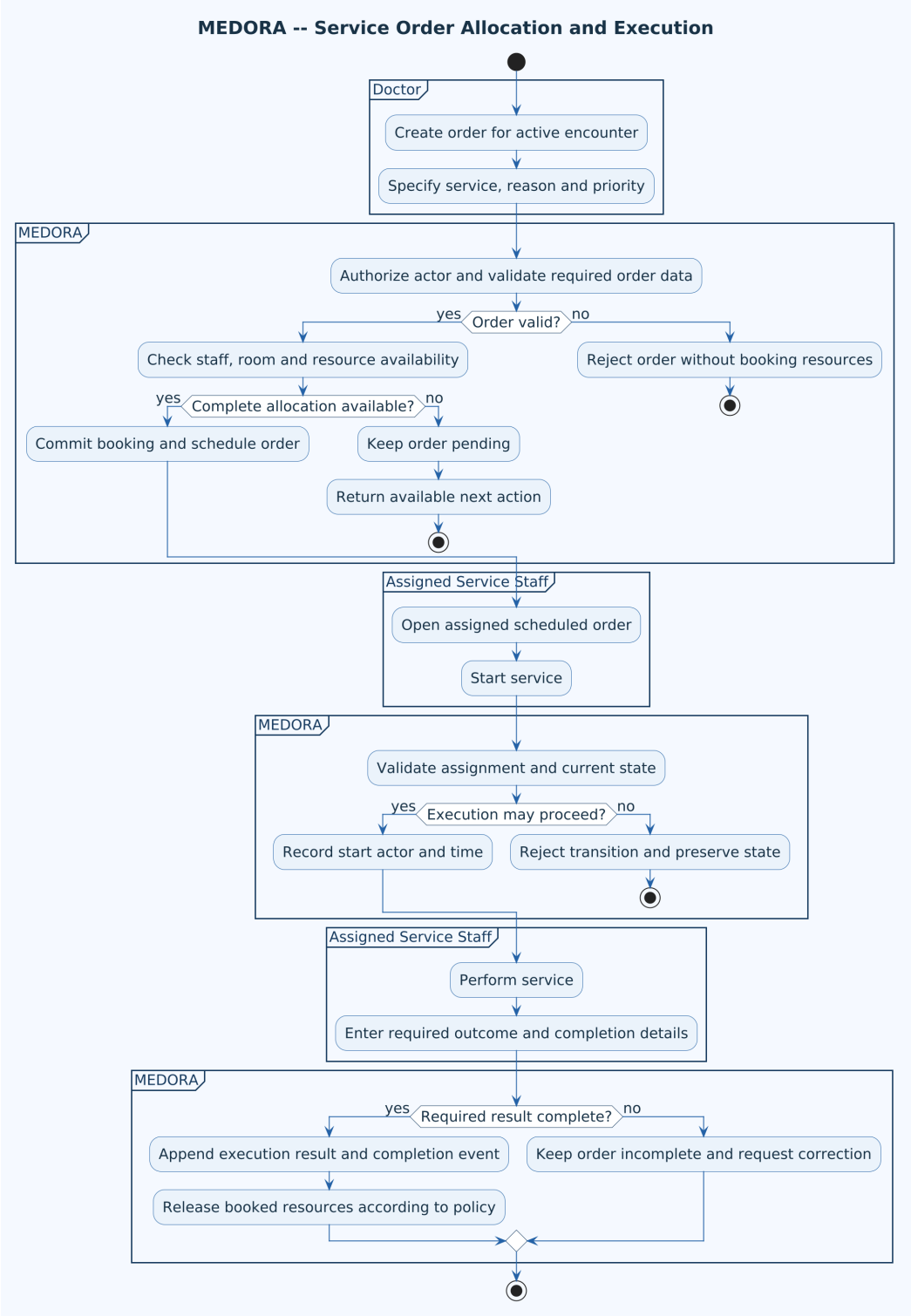


Figure 21: Activity model for service-order allocation and execution

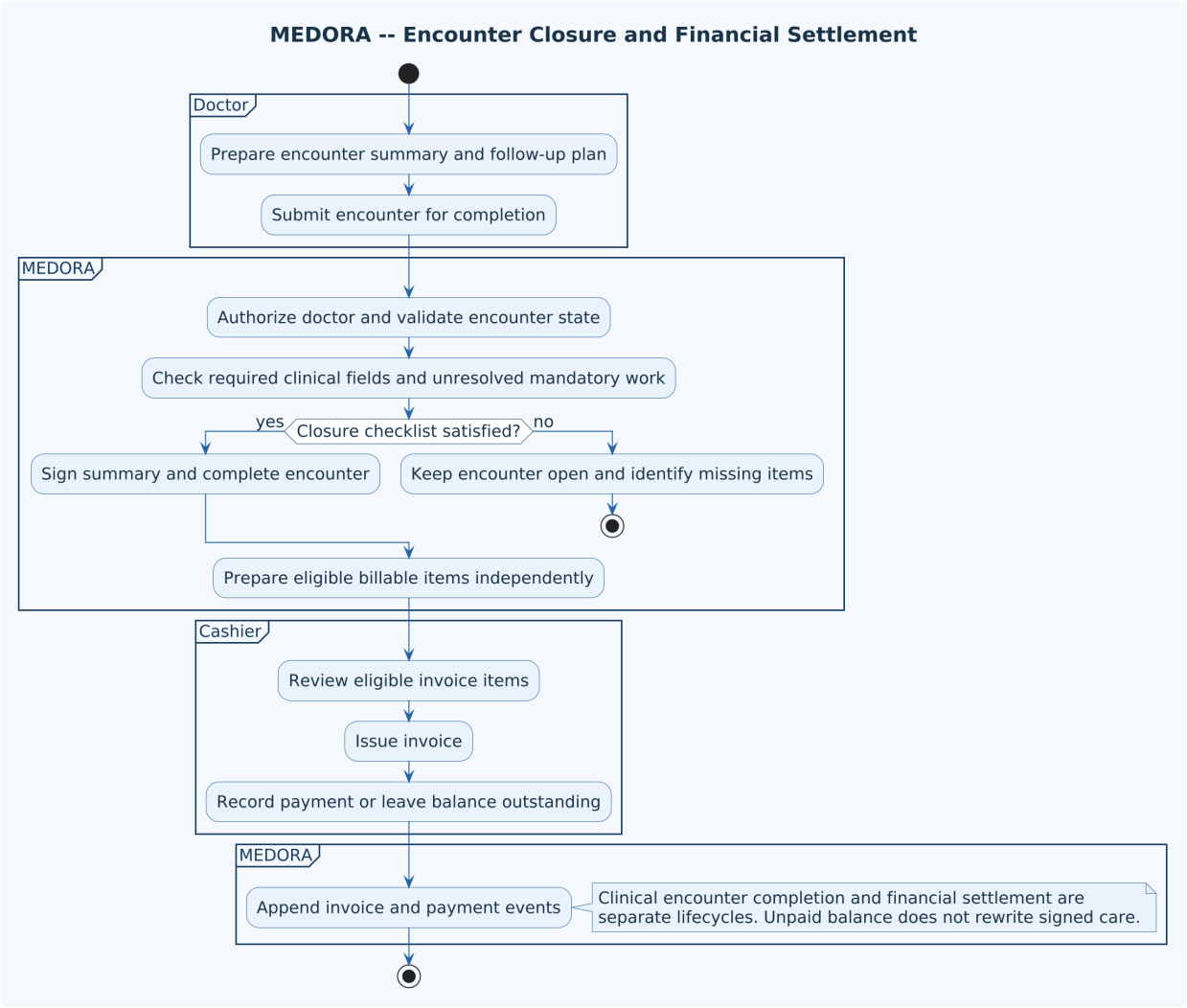


Figure 22: Activity model for encounter closure and independent financial settlement

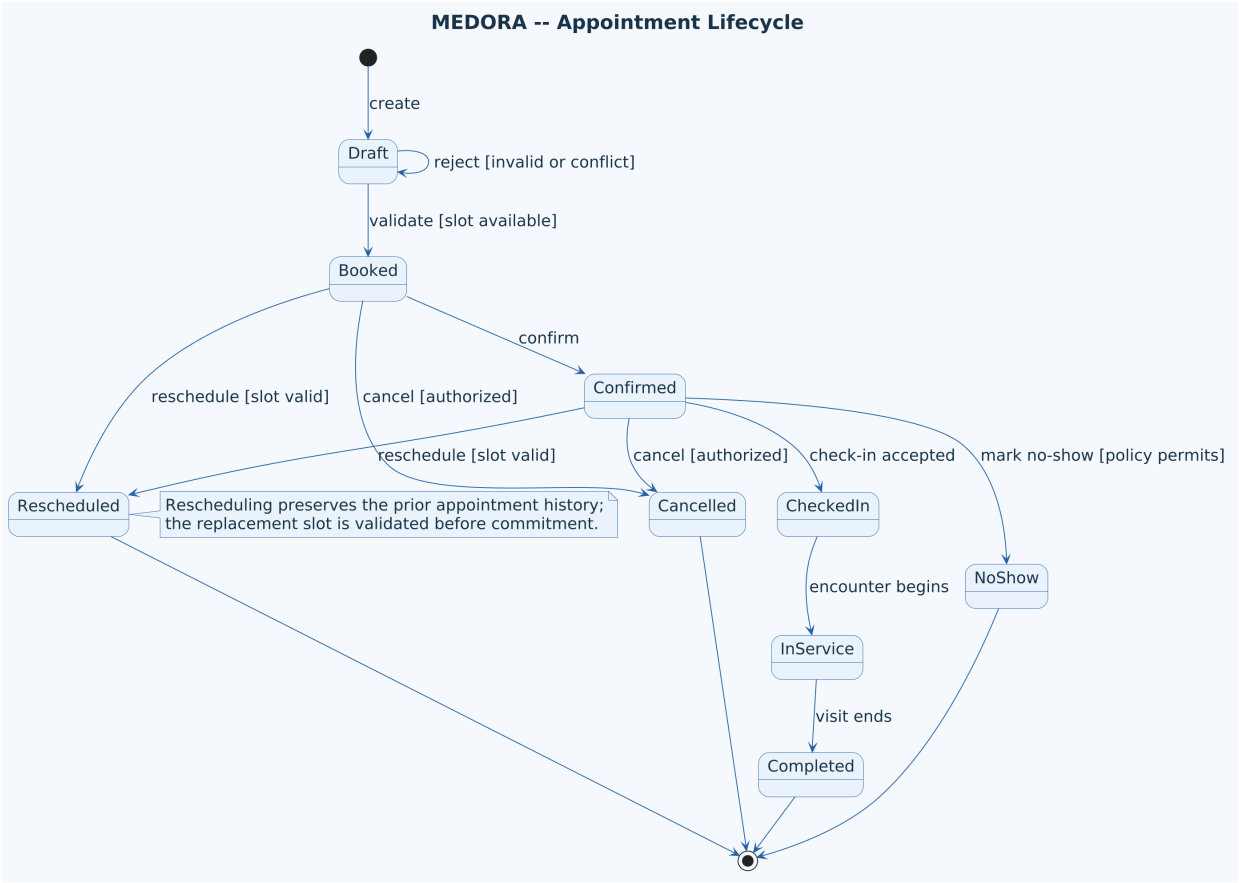


Figure 23: Proposed appointment lifecycle and terminal outcomes

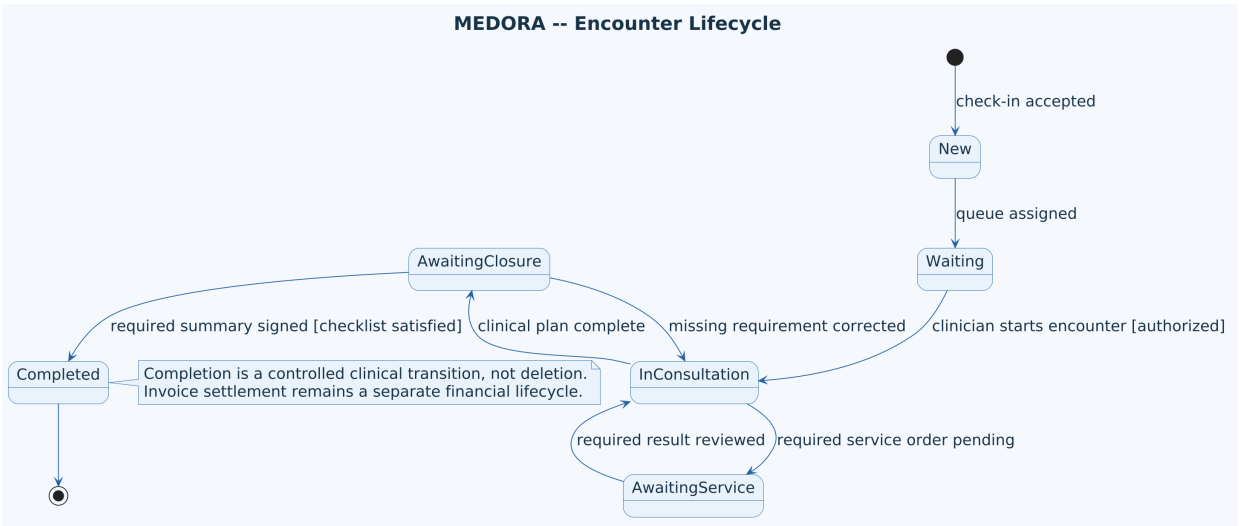


Figure 24: Proposed encounter lifecycle from check-in to clinical completion

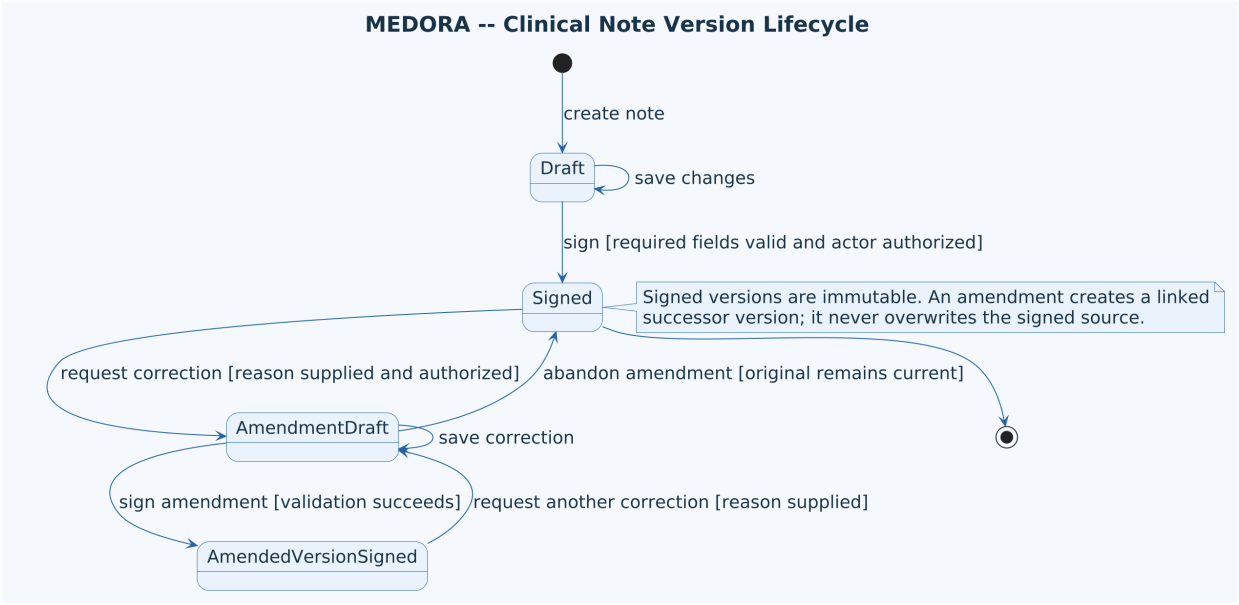


Figure 25: Clinical-note version lifecycle with immutable signed versions

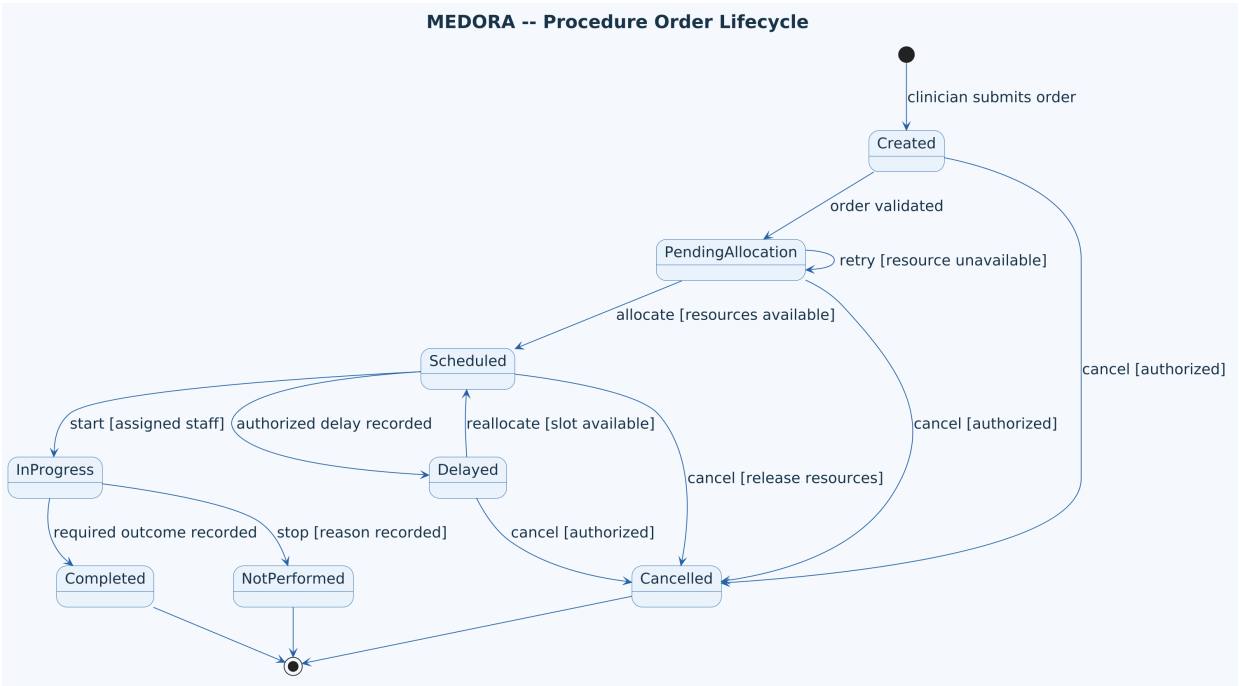


Figure 26: Proposed procedure-order lifecycle with allocation and execution outcomes

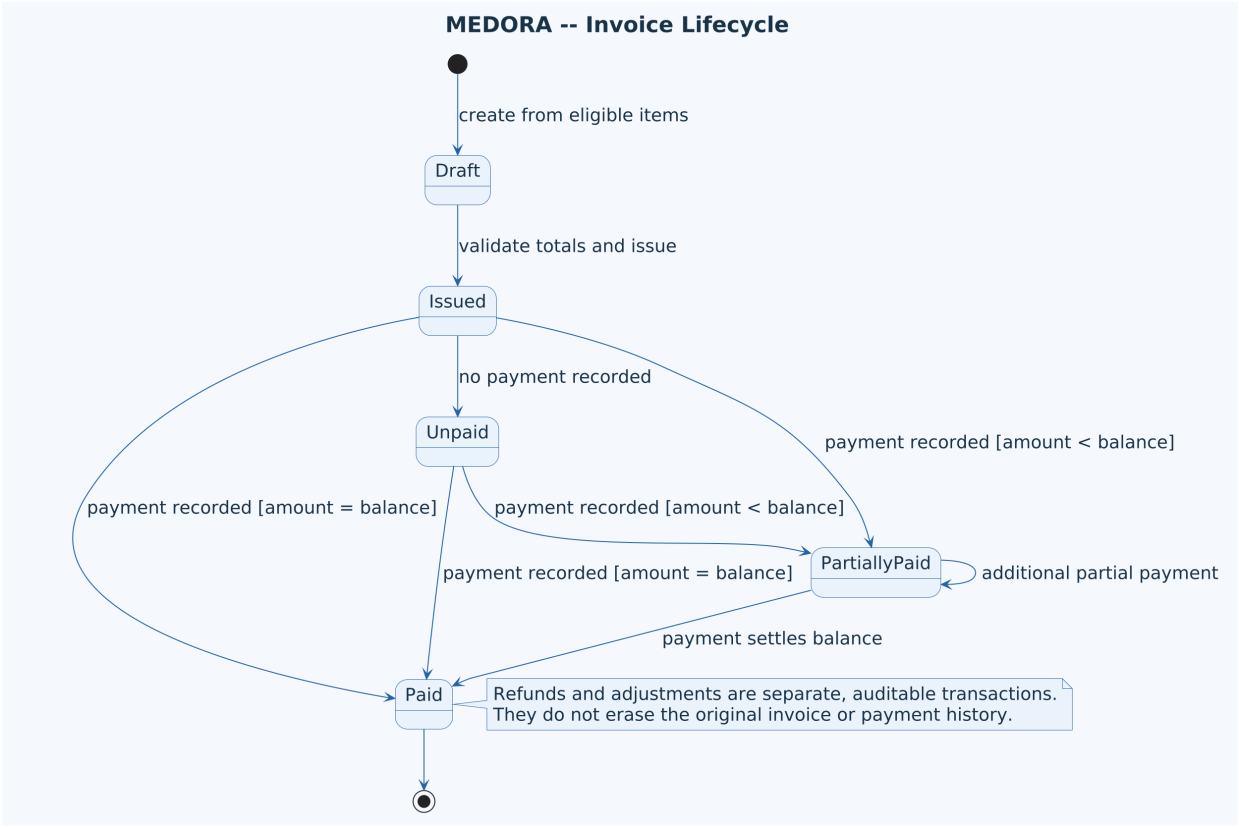


Figure 27: Proposed invoice lifecycle for unpaid, partial-payment, and paid states

5.2 Source Code Organization and Module Structure

The repository contains no application code yet. To make the design actionable, this section proposes a TypeScript modular-monolith implementation baseline: Next.js for the role-specific web client, a NestJS REST API, PostgreSQL persistence through Prisma migrations, private object storage for attachments, and Docker Compose for local development. This stack is a recommendation, not an approved or implemented technology decision; the team should record approval as ADR-001 before scaffolding. The API remains the sole authorization and state-transition boundary; the browser must not access persistence directly.

Listing 1: Proposed source layout for the first implementation

```
medora/
    apps/
        web/                          # Next.js role-specific browser UI
        api/src/modules/
            identity/ patients/ scheduling/
            encounters/ prescriptions/ orders/
            finance/ governance/ audit/
    packages/
        contracts/                    # versioned DTOs and OpenAPI
           contract
        persistence/                  # Prisma schema, repositories,
           migrations
```

```
infra/
    compose.yaml                  # local-only services and test
       dependencies
    proxy/                        # non-production HTTPS configuration
tests/
    unit/ integration/ e2e/       # synthetic data only
doc/asset/                       # report and editable UML sources
```

Each API module owns its use cases and validation. Shared contracts contain request/response types, not business rules. Persistence migrations are reviewed and versioned; domain modules call repositories through application services. Authorization, transaction boundaries, audit emission, and idempotency belong in the API/application layer, not in UI visibility rules. A later implementation may reorganize this layout only with an updated module mapping and traceability review.

Evidence item	Repository observation	Status / implication
Application source	`src/README.md` is a placeholder asking for source code.	No application modules or implemented behavior are evidenced.
Build/dependency manifest	No application package manifest or runtime build configuration is present.	Next.js/NestJS/TypeScript is the recommended ADR-001 baseline, pending team approval.
Persistence and API	No database schema, migration, API contract, or service implementation is present.	PostgreSQL/Prisma and the versioned REST contract are proposals, not implementation facts.
Behavioral assets	PlantUML sources and rendered figures are stored under `doc/asset/05-behavioral-modeling-implementation/`.	These are proposed behavioral models; they are not executable application code.
Deployment	No container, server, environment, or deployment configuration is present.	Production topology, hosting, and operational controls are not yet determined.

Table 20: Implementation and deployment evidence available in the repository

The source layout above maps the report's logical layers and functional groups to a buildable project structure while preserving the current evidence boundary. No listed directory, package, or module should be described as implemented until it exists and is reviewed in the repository.

5.3 Implementation of Core Functionalities

The following implementation slices turn the requirements into an ordered build plan. All are currently **Not Implemented**; the invariant column defines the minimum behavior each slice must preserve before it can be accepted.

Implementation slice	Trace	Acceptance invariant	Status
Identity and authorization	FR-001–FR-004; NFR-06	Server rejects out-of-scope reads/writes; patient duplicate review precedes merge.	Not Implemented
Scheduling and reception	FR-005–FR-006, FR-017	Concurrent conflicts do not commit; accepted check-in creates one active encounter.	Not Implemented
Encounter and clinical records	FR-007–FR-010, FR-020, FR-032	Signed versions are immutable; encounter close validates required work.	Not Implemented
Prescription and service orders	FR-011–FR-019	Configured safety checks are bounded; orders, bookings, and actual events remain distinct.	Not Implemented
Finance and reporting	FR-022–FR-027	Price is snapshotted; payment/refund are idempotent append-only events.	Not Implemented
Governance and attachments	FR-028–FR-031; NFR-07	Sensitive actions are scoped and audited; files are private and access-controlled.	Not Implemented

Table 21: Proposed implementation slices and acceptance invariants

5.4 Development and Deployment Environment

Table 22 defines a reproducible target environment without implying any environment currently exists. Secrets, patient data, and production records must never be placed in the local development stack. Production hosting, availability design, data residency, backup retention, and incident ownership require explicit approval.

Environment	Proposed services and configuration	Data / release controls
Local development	Node.js LTS toolchain; web and API; PostgreSQL; private S3-compatible storage emulator; Docker Compose.	Synthetic fixtures only; local secrets in ignored environment files; migrations applied from source.
Automated test	Ephemeral database and private file store; isolated test accounts; unit, integration, and Playwright end-to-end suites.	No real patient data; reset between runs; deterministic seeds and retained test logs.
Staging	One controlled application deployment behind HTTPS reverse proxy; restricted database and private attachment store; monitoring and backup/restore rehearsal.	De-identified/synthetic data only; role access reviewed; release build and migration version recorded.
Production target	Hosting provider and topology TBD after capacity, privacy, recovery, and cost review; separate app, database, object storage, and protected backup boundaries.	TLS, managed secrets, least-privilege service accounts, audit retention, restore exercises, and approved operational ownership.

Table 22: Proposed development and deployment environment matrix

The proposed environment matrix above captures the intended trust boundaries without selecting a vendor topology. The local LaTeX/PlantUML/Kroki tools used to produce this report are documentation tooling and are not part of the MEDORA runtime.

5.5 Proposed User Interfaces and API Evidence

No UI/API screenshots or proposed wireframe images are included because there is no running application. WorkStation Section 7 provides a screen inventory that can guide later UX design; Appendix C provides proposed API/data examples, not captured responses from an existing endpoint. A future implementation evidence package must pair a redacted screen/API capture with build ID, actor role, workflow state, and test-case ID.

Evidence required before implementation claims

After implementation, capture redacted UI, API, and workflow evidence from a running non-production build. Record build/version, actor role, workflow state, and linked test case. API evidence should include the approved endpoint/schema, authorization context, sanitized request/response, and observed result. Never include real patient data or secrets.

6 System Testing and Evaluation

6.1 Testing Strategy and Test Environment

Testing is planned around the business risks and acceptance criteria in WorkStation and Sections 2–3. The strategy combines workflow, authorization, integrity, concurrency, and quality-attribute verification; it does not treat a rendered diagram or a successful report build as evidence that the application works.

The repository contains no application under test, database, API, automated test suite, test dataset, or deployment environment. Consequently, the test environment is not yet established and execution is blocked. When an application exists, testing should use an isolated non-production environment, synthetic patient data, controlled user roles, reproducible configuration, and a recorded build identifier. Performance and recovery checks additionally require an agreed workload and a provisioned environment representative of the intended deployment.

Test level	Planned purpose	Current evidence
Unit / domain	Validate lifecycle guards, calculations, authorization decisions, and invariants.	Not available; no implementation or unit tests.
Integration	Verify transactions across encounter, order, resource, invoice, payment, and audit boundaries.	Not available; no database or service integrations.
End-to-end workflow	Exercise the reception-to-encounter, clinical, service, and financial paths by role.	Not available; no running application or test environment.
Security / negative access	Send unauthorized requests directly to protected server operations and verify safe rejection.	Not available; no API or identity implementation.
Quality attributes	Measure response time, accessibility, locale behavior, availability, and restore objectives.	Not available; targets remain proposed and unmeasured.

Table 23: Planned verification levels and present evidence boundary

6.2 Test Scenarios and Test Case Design

The following test cases translate high-risk requirements and use cases into executable checks. Expected outcomes are specifications for future tests; the status column records that none has been executed.

Table 24: Planned test cases and current execution status

ID	Scenario and expected result	Trace	Status
TC-001	Call a protected patient/clinical operation without the required role or scope; reject the request without returning protected data or applying a state change.	FR-001–FR-002; NFR-06; UC-001	Not Run
TC-002	Register a patient matching an existing identity; flag a possible duplicate and prevent an unauthorized merge.	FR-003–FR-004; BR-001; UC-010–UC-018	Not Run
TC-003	Submit concurrent bookings for the same constrained provider, room, or resource; accept no conflicting allocation and preserve rejected requests without a booking.	FR-005, FR-015, FR-017; NFR-15; UC-021–UC-026	Not Run
TC-004	Check in an accepted walk-in and attempt a second active encounter for the same patient; create one valid encounter/queue and reject the duplicate.	FR-006; BR-003; UC-027–UC-030	Not Run
TC-005	Sign a note with missing required content, then amend a signed note; block incomplete signing and retain the original signed version with a linked, reasoned amendment.	FR-009, FR-020, FR-032; NFR-07; UC-037–UC-042	Not Run
TC-006	Issue a prescription with a configured safety warning; require the permitted response/override evidence and verify issue alone creates no dispensing or administration event.	FR-011–FR-013; BR-006, BR-008; UC-043–UC-052	Not Run
TC-007	Allocate a service order with unavailable or conflicting resources, then complete a valid execution; do not commit a conflict and do not mark complete without required results.	FR-014–FR-018; NFR-15; UC-039–UC-040	Not Run
TC-008	Record partial and repeated payment submissions; maintain the correct balance and prevent a duplicate request from applying a second financial effect.	FR-022–FR-023; NFR-15; UC-060–UC-069	Not Run
TC-009	Request a refund with invalid authority, amount, or reason; reject or hold it for approved review and retain the original transaction history.	FR-023, FR-030; BR-009; UC-065–UC-066	Not Run
TC-010	Export a scoped report and review its audit evidence; enforce the actor's scope and record the controlled export event.	FR-026–FR-027, FR-030; NFR-06–NFR-07; UC-076–UC-079	Not Run

ID	Scenario and expected result	Trace	Status
TC-011	Measure ordinary-screen and search response times under an agreed representative workload; compare p95 values with the provisional targets.	NFR-01; WorkStation Section 10	Not Run
TC-012	Restore a protected backup and measure data loss and recovery time against approved RPO/RTO; a backup log alone is insufficient evidence.	NFR-02–NFR-03; WorkStation Section 10	Not Run
TC-013	Verify keyboard operation, labels, contrast, Vietnamese text, configured date/time, and currency formats on supported browsers.	NFR-10–NFR-11, NFR-14	Not Run

6.3 Functional Testing and Use Case Coverage

The planned cases cover representative paths across identity/access, scheduling/reception, clinical documentation, prescribing, service execution, finance, and governance. They are a first execution set, not proof of exhaustive FR-001–FR-034 coverage. Before implementation acceptance, each requirement and detailed use case must be mapped to one or more test cases, including negative authorization and exception cases.

Requirement group	Representative tests	Coverage status
FR-001–FR-004, FR-028–FR-031	TC-001–TC-002, TC-010	Specified; implementation and execution not evidenced.
FR-005–FR-006, FR-017	TC-003–TC-004	Specified; implementation and execution not evidenced.
FR-007–FR-010, FR-020–FR-021, FR-032	TC-004–TC-005	Specified; implementation and execution not evidenced.
FR-011–FR-019	TC-006–TC-007	Specified; implementation and execution not evidenced.
FR-022–FR-027, FR-030	TC-008–TC-010	Specified; implementation and execution not evidenced.
NFR-01–NFR-16	TC-001, TC-003, TC-005–TC-013	Verification methods proposed; no measurements or results available.

Table 25: Representative functional and quality-requirement test coverage

6.4 Exception Handling, Authorization, and Data Integrity Testing

The highest-risk checks are rejection paths that must leave no invalid partial state: unauthorized access (TC-001), duplicate identity/encounter handling (TC-002 and TC-004), concurrent resource allocation (TC-003 and TC-007), incomplete signing and append-only amendments (TC-005), bounded prescription overrides (TC-006), and idempotent payment/refund handling (TC-008–TC-009). Expected behavior is specified, but the corresponding server-side controls and transactional boundaries cannot be verified until they exist in code.

6.5 Test Execution Results and Evidence

Execution status: Not Run

As of this report revision, there is no runnable MEDORA application, test harness, database, or configured test environment in the repository. All 13 cases in Table 24 are therefore **Not Run**. No case is reported as Pass or Fail, no coverage percentage is claimed, and no execution screenshot or log is available. Execution is blocked pending an implemented build and test environment.

Status	Count	Evidence
Pass	0	No executed results.
Fail	0	No executed results.
Blocked	0	The suite has not started; the prerequisite gap is recorded below.
Not Run	13	No system under test or execution environment.

Table 26: Test execution summary at report time

6.6 Evaluation Against Functional and Non-functional Requirements

Functional requirements are specified and traced to use cases, but none is verified as implemented. The quality targets below come from WorkStation and remain proposed acceptance criteria pending owner confirmation and measurement.

Evaluation measure	Proposed acceptance basis	Current evaluation
Registration-to-completion time	Reduce median by at least 25% against a documented baseline for the representative scenario.	Not measured; no baseline or executable workflow.
Conflicting allocations	Zero accepted conflicts in all defined concurrency cases.	Not measured; no scheduling implementation.
Encounter completeness	At least 95% of configured mandatory fields for tested completed visits.	Not measured; no configured forms or visits.
Sensitive-event traceability	100% of tested sensitive events have actor, time, action, and target evidence.	Not measured; no audit implementation.
Authorization	100% of negative authorization cases rejected at the server/API boundary.	Not measured; no API or access-control implementation.
Response time	p95 ordinary screens $\leq$ 2 seconds and search $\leq$ 3 seconds under an agreed workload.	Proposed only; no deployment or load test.
Availability and recovery	Provisional 99.5% monthly availability, RPO $\leq$ 15 minutes, and RTO $\leq$ 4 hours.	Proposed only; no operational environment or restore exercise.

Table 27: Evaluation criteria and evidence status

6.7 Limitations and Outstanding Issues

The principal evaluation limitation is absence of an implemented system. The proposed test cases cannot establish behavior, performance, security, accessibility, recovery, or user acceptance. The clinic must also confirm clinical templates and warning sources, retention and consent rules, merge/refund/override approvals, branch scope, and patient-portal release policy before those behaviors can be treated as definitive acceptance requirements. No legal or certification compliance is claimed.

7 Conclusion and Future Work

7.1 Summary of Achievements

The current deliverable establishes a clinic-focused business and requirements baseline, a traceable use-case model, and a set of behavioral models for appointment booking, check-in, clinical-note integrity, prescription safety, service execution, encounter closure, and payment history. It also defines a practical test plan with expected outcomes and explicit status reporting. These are analysis and design achievements; they do not demonstrate that MEDORA has been implemented, deployed, or tested.

7.2 Limitations of the Current System

There is no application implementation in the repository, so MEDORA should be described as a specified/proposed system rather than an operational product. No source modules, persistent schema, API, user interface, deployment topology, test execution evidence, or user feedback are available. The use-case, activity, sequence, and state diagrams express intended behavior; they remain subject to policy confirmation and later reconciliation with code. Performance, availability, recovery, and success-rate values are targets only. Clinical rule sources, data retention, consent, approval thresholds, multi-branch operation, and portal publication remain open decisions.

7.3 Future Improvements and Extensions

Future work should proceed in evidence-producing stages:

1. Confirm clinic policies for authorization, clinical safety, record correction, finance, retention, and lifecycle transitions.
2. Decide and document the architecture, persistence model, deployment assumptions, and mapping from approved components to source modules.
3. Implement a testable MVP for identity, appointment/check-in, encounter documentation, role/scope enforcement, and audit evidence.
4. Extend the MVP with clinician-approved prescription and service workflows, then basic finance with separate, immutable transaction history.

5. Establish automated and negative-authorization tests; measure quality targets using synthetic data and a recorded environment; capture privacy-redacted UI/API evidence and usability/recovery results.

6. Add the patient portal or external integrations only after scope, contracts, security controls, and operational ownership are approved.

This sequence keeps the report aligned with the project's clinic scope and makes each future claim auditable against source code, test results, or operational evidence.

A Detailed Use Case Specifications

The following specifications expand the highest-risk use cases from Section 3. They are the proposed behavior baseline for implementation and verification. Facility policies, clinical rules, approval thresholds, and retention requirements must be approved before the corresponding behavior is considered final.

A.1 UC-021 Book Appointment

Goal Reserve a valid clinic service slot for an identified patient.

Primary actor
Receptionist. Patient self-service is Phase 2.

Trigger A patient requests an appointment or a receptionist schedules follow-up.

Preconditions
Actor is authenticated and facility-scoped; patient identity and requested service are resolved; scheduling data is available.

Main success flow
1. Receptionist selects patient, service, provider, and requested time.
2. System validates request fields, actor permissions, facility, and patient scope.
3. System checks provider schedule, duration, room, and any configured constrained resource.
4. System repeats the availability check at transaction commit and atomically creates the Appointment and required ResourceBooking records.
5. System records actor/time and returns appointment ID and status.

Alternative flows
If a slot is unavailable, the system offers permitted alternatives; the receptionist selects one and the system repeats validation. A draft request may be retained without reserving resources if the clinic enables drafts.

Exception flows

Invalid identity, unauthorized scope, unavailable resource, or a concurrent conflict rejects the request without partial appointment/resource records. Repeating a successful request with the same idempotency key returns the original result.

Postconditions and data

Success creates one Appointment and optional resource allocation; it does not create an Encounter. Failure leaves the prior schedule unchanged.

Authorization

Receptionist in facility scope; any patient portal actor is separately authenticated and limited to their own identity.

Trace BR-002; FR-005, FR-015, FR-017; UC-020–UC-026; TC-003; Figures 15 and 23.

A.2 UC-027 Check In Patient

Goal Start one valid outpatient Encounter and place it in the configured queue.

Primary actor

Receptionist.

Trigger An identified booked patient arrives or a walk-in is accepted by facility policy.

Preconditions

Receptionist is authenticated and scoped; patient identity is resolved; walk-in policy is known where applicable.

Main success flow

1. Receptionist selects an Appointment or accepted walk-in intake.
2. System validates patient identity, facility scope, and current Appointment state.
3. System checks whether the patient already has an active Encounter under the approved rule.
4. System creates/activates the Encounter and assigns the configured queue state atomically.
5. System returns Encounter and queue references.

Alternative flows

Accepted walk-ins use the same identity and duplicate-Encounter checks. A rejected walk-in remains unregistered as an active Encounter and receives the permitted next step.

Exception flows

Unresolved identity, unauthorized facility, invalid Appointment state, or a duplicate active Encounter rejects the transition without creating a second Encounter.

Postconditions and data

Success links Encounter to one Patient and optionally one Appointment. A booking without check-in remains an Appointment only.

Authorization

Receptionist may create/check in; Nurse may read only the queue assigned by role/facility policy.

Trace BR-003; FR-006; UC-027–UC-030; TC-004; Figures 16 and 24.

A.3 UC-037/UC-038 Sign and Amend Clinical Note

Goal Finalize a clinically attributable note while preserving any later correction history.

Primary actor

Doctor assigned to the active Encounter.

Trigger Doctor completes documentation or identifies an error in signed content.

Preconditions

Doctor is authenticated, authorized for the Encounter, and the encounter is in a state that permits documentation.

Main success flow

1. Doctor saves a draft with the required clinical sections.
2. System validates required data, assignment, and expected current version.
3. Doctor requests signing.
4. System records an immutable signed version with actor and timestamp and emits audit evidence.
5. System returns the signed-version reference.

Alternative flows

An incomplete note remains a Draft. A correction creates a successor amendment linked to the signed predecessor; the Doctor supplies a reason and signs the amendment separately.

Exception flows

Missing fields, stale version, invalid assignment, or unauthorized amendment rejects the operation. The signed source and all previous versions remain unchanged.

Postconditions and data

NoteVersion history is append-only; no direct update of signed content is allowed.

Authorization

Assigned Doctor may sign/amend; all reads/writes remain role-, facility-, and encounter-scoped.

Trace BR-005; FR-009, FR-020, FR-032; UC-037/UC-038; TC-005; Figures 17 and 25.

A.4 UC-048 Validate and Issue Prescription

Goal Issue an authorized, complete prescription after configured safety checks.

Primary actor

Doctor assigned to the active Encounter.

Trigger Doctor submits a prescription draft for issue.

Preconditions

Patient and Encounter are verified; Doctor is assigned and authorized; at least one medicine item is present.

Main success flow

1. Doctor submits medicine items and directions.
2. System validates required fields and configured rule inputs that are available.
3. System presents any supported warning and its reason.
4. Doctor resolves the warning or performs a permitted override with reason.
5. System issues the Prescription, records actor/time/audit evidence, and returns the issue reference.

Alternative flows

Doctor may keep the Prescription in Draft. If no blocking warning is raised, the Doctor may issue after required-field validation.

Exception flows

Missing data, a blocking warning, unavailable required policy data, unauthorized override, or stale Encounter state prevents issue and creates no administration event.

Postconditions and data

Issued Prescription and PrescriptionItems are distinct from MedicationAdministration; issuing alone never asserts medicine was dispensed/administered.

Authorization

Assigned Doctor issues; medicine staff record only the separately authorized administration/dispensing event.

Trace BR-006/BR-008; FR-011–FR-013; UC-043–UC-052; TC-006; Figure 18.

A.5 UC-039/UC-040 Schedule and Execute Service Order

Goal Coordinate a requested service with valid resources and record its actual outcome.

Primary actors

Doctor creates the order; assigned Service Staff records execution.

Trigger Doctor adds a service/procedure to an active Encounter.

Preconditions

The Encounter is active; service catalogue and resource requirements are configured; actors have relevant scope.

Main success flow

1. Doctor records service, reason, priority, and required preparation.
2. System validates Encounter, order fields, and actor permissions.
3. Scheduler requests a time/resource allocation.
4. System validates room, staff, and constrained-resource availability at commit.
5. Assigned staff starts the order and records actor/time.
6. Staff records required outcome; system validates and marks execution complete.

Alternative flows

If no resource slot is available, the order remains pending without a booking and may be rescheduled after a fresh availability check. A delayed order remains non-complete.

Exception flows

Conflict, missing assignment, invalid state, or incomplete outcome rejects the transition and preserves the order's last valid state. Cancellation records a reason and releases resources according to policy.

Postconditions and data

ProcedureOrder, ResourceBooking, and execution outcome are linked but distinct records.

Authorization

Doctor creates order; assigned Service Staff updates execution; scheduling authority follows clinic policy.

Trace BR-002/BR-007; FR-014–FR-018; UC-039/UC-040; TC-007; Figures 19 and 21.

A.6 UC-063/UC-065 Record Payment and Refund

Goal Record settlement or refund while preserving a complete financial history.

Primary actors

Cashier posts payment; authorized Finance Staff posts refund/adjustment.

Trigger Eligible Invoice is issued or a prior PaymentTransaction requires refund/adjustment.

Preconditions

Actor is authenticated and facility/finance-scoped; invoice and original transaction state are known.

Main success flow

1. Cashier selects invoice and enters amount, currency, and method.
2. System validates balance, state, scope, and idempotency key.

3. System appends PaymentTransaction and returns the resulting invoice balance.
4. For a refund, Finance Staff references the original transaction, supplies a reason, and obtains configured approval.
5. System appends a refund/adjustment transaction and audit event; the original payment remains intact.

Alternative flows

Partial payment leaves an outstanding balance. Same-key retries return the original result.

Exception flows

Invalid amount/state, conflicting idempotency key, insufficient authority, or missing reason/approval has no financial effect.

Postconditions and data

Invoice and PaymentTransaction history is preserved; clinical records are unchanged.

Authorization

Cashier posts payments; a separately authorized finance role handles refunds and approvals.

Trace BR-009; FR-022/FR-023; UC-063/UC-065; TC-008/TC-009; Figure 20.

B Supplementary UML Diagrams

The editable sources, rendered figures, and traceability inventory are catalogued in Table 28. The diagrams are proposed models and are not implementation evidence. Structural diagrams for architecture, components, classes, and ERDs are in Section 4; workflow and lifecycle diagrams are in Section 5.

Table 28: UML source and rendered-figure inventory

Source group	Types / count	Purpose	Status
Section 2 business analysis	Activity 2; domain 1	Outpatient journey, appointment/check-in exceptions, and clinical domain concepts.	Rendered
Section 3 requirements	Functional 1; overall use case 1; focused use case 9	Capability groups, system boundary, actors, and detailed actor goals.	Rendered
Section 4 architecture	Reserved for Chapter 4 owner	This contribution restores the original Chapter 4 outline; no Chapter 4 UML is claimed here.	Pending
Section 4 persistence	Reserved for Chapter 4 owner	ERD and schema artifacts remain with the Chapter 4 contributor.	Pending

Source group	Types / count	Purpose	Status
Section 4 classes	Reserved for Chapter 4 owner	Structural class artifacts remain with the Chapter 4 contributor.	Pending
Section 5 interaction	System sequence 6	Actor-visible success, alternative, and rejection outcomes for high-risk use cases.	Rendered
Section 5 process/lifecycle	Activity 2; state machine 5	Service execution, closure/finance separation, and entity transitions.	Rendered
Section 5 UI evidence	WorkStation screen inventory	No UI images are included; capture actual screens after implementation.	Not available

The PlantUML source is stored in the matching `uml_code/` folder and the rendered PNG in `figure/`. Parsing and rendering validate syntax and support visual review; they do not establish implementation, clinical approval, or user acceptance.

C API Specifications and Data Schemas

API request/response examples and a common error schema are proposed in this appendix and Chapter 5. The examples below provide representative persisted data shapes for implementation planning. All IDs and values are synthetic.

C.1 Representative Appointment Data

Listing 2: Proposed Appointment resource shape

```
{
  "appointmentId": "appt-demo-1001",
  "patientId": "pat-demo-001",
  "serviceId": "svc-demo-012",
  "providerId": "emp-demo-004",
  "facilityId": "facility-demo-main",
  "startAt": "2026-10-20T09:30:00+07:00",
  "endAt": "2026-10-20T10:00:00+07:00",
  "status": "BOOKED",
  "createdAt": "2026-10-16T09:00:00+07:00"
}
```

C.2 Representative Encounter Data

Listing 3: Proposed Encounter resource shape

```
{
  "encounterId": "enc-demo-2001",
  "patientId": "pat-demo-001",
  "appointmentId": "appt-demo-1001",
  "status": "WAITING",
  "facilityId": "facility-demo-main",
  "assignedClinicianId": "emp-demo-004",
  "startedAt": "2026-10-20T09:25:00+07:00"
}
```

C.3 Representative Audit Event Data

Listing 4: Proposed audit event shape with no clinical payload duplication

```
{
  "auditEventId": "audit-demo-9001",
  "actorId": "user-demo-001",
  "action": "CLINICAL_NOTE_SIGNED",
  "targetType": "ClinicalNoteVersion",
  "targetId": "note-version-demo-002",
  "facilityId": "facility-demo-main",
  "occurredAt": "2026-10-20T10:02:00+07:00",
  "reason": null,
  "correlationId": "req-demo-001"
}
```

These shapes are illustrative contract baselines. The approved OpenAPI schema, database types, retention rules, field-level access policy, and migration definitions must be authored and reviewed before implementation.

D Detailed Test Cases and Execution Results

Detailed test procedures and expected outcomes are listed in Table 29. The actual-result, tester, execution timestamp, build identifier, and evidence fields are intentionally not populated: all 13 cases remain **Not Run** because no application or test environment exists. No pass/fail result is claimed.

Table 29: Detailed test procedures and expected outcomes

ID	Setup	Procedure	Expected result / evidence to capture	Status
TC-001	Synthetic patient; authenticated user without clinical permission.	Call protected patient/clinical read and write endpoints directly, bypassing the UI.	Reject both requests; disclose no protected fields; create no mutation; retain sanitized response and audit-policy evidence.	Not Run
TC-002	One synthetic patient; registrar and ordinary receptionist accounts.	Register a matching identity, then request merge as receptionist.	Flag possible duplicate; deny unauthorized merge; preserve source records and history.	Not Run

ID	Setup	Procedure	Expected result / evidence to capture	Status
TC-003	One provider/room/resource slot; two concurrent clients.	Submit conflicting bookings with separate idempotency keys at the same time.	At most one booking commits; losing request receives conflict/alternatives; no orphan resource reservation remains.	Not Run
TC-004	Patient with no active encounter and an accepted appointment or walk-in.	Check in once; repeat check-in while the first encounter remains active.	First call creates one encounter/queue assignment; second is rejected without duplicate state.	Not Run
TC-005	Assigned doctor; incomplete draft and then corrected note.	Attempt signing while required content is missing; correct, sign, then append an amendment with reason.	Incomplete signing is blocked; signed source remains immutable; amendment links predecessor and records actor/time/reason.	Not Run
TC-006	Synthetic, clinician-approved safety-rule fixture; doctor and medicine staff roles.	Issue with a warning; test dismissal and permitted override; inspect medication events.	Apply configured warning behavior; require authorized override reason; prescription issue alone creates no administration event.	Not Run
TC-007	Valid order and two requests competing for the same resource.	Attempt conflicting allocations; submit incomplete execution result and then a valid result.	Conflict does not commit; incomplete result cannot complete order; valid result records performer/time and releases resources by policy.	Not Run
TC-008	Issued invoice with known total; cashier in authorized facility.	Post partial payment; retry same key; attempt another payment with a new key.	Same key returns original event; permitted new payment updates balance exactly once; reconciliation matches transaction history.	Not Run
TC-009	Paid synthetic invoice; cashier and authorized finance role.	Request refund with unauthorized role, invalid amount, missing reason, then valid approval.	Invalid requests have no financial effect; valid refund appends event and preserves original payment.	Not Run
TC-010	Two facility scopes with synthetic report data.	Query and export through auditor/manager roles across authorized and unauthorized scopes.	Enforce scope; audit actor, time, filter, and target; do not duplicate protected clinical content in logs.	Not Run
TC-011	Staging-like environment, agreed synthetic dataset, representative concurrent load.	Measure ordinary screens and search; record sample size, p95, build, and configuration.	Compare results with owner-approved targets; retain sanitized measurements; do not infer production capacity.	Not Run
TC-012	Encrypted synthetic backup and clean recovery environment.	Restore database and private files; verify checksums and measure data loss and recovery time.	Compare measured RPO/RTO with approved objectives; retain restore log and timestamps.	Not Run
TC-013	Supported browsers and synthetic Vietnamese names, dates, currencies; keyboard-only user.	Complete core workflows using keyboard; review labels, errors, contrast, date/time, and currency formats.	Record browser/version and findings; no blocking accessibility or localization defect remains at acceptance.	Not Run

For an executed run, add actual result, tester, timestamp, build ID, environment, evidence link, and defect ID if applicable. A test may then be marked Pass, Fail, or Blocked. Until those fields are supported by real execution evidence, the status remains Not Run.

References

Thi Thanh Thuy Nguyen et al. Waiting time in the outpatient clinic at a national hospital in vietnam. *BMC Health Services Research*, 18:1–9, 2018. doi: 10.1186/s12913-018-3285-2. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC5995730/.

Thi Thanh Thuy Nguyen et al. Outpatient waiting time at vietnam health facilities: policy implications for medical examination procedure. *Healthcare*, 8(1):63, 2020. doi: 10.3390/healthcare8010063. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC7151016/.

Organisation for Economic Co-operation and Development. *Health at a Glance 2023: OECD Indicators*. OECD Publishing, Paris, 2023. doi: 10.1787/7a7afb35-en. URL https://www.oecd.org/en/publications/health-at-a-glance-2023_7a7afb35-en/full-report/digital-health-at-a-glance_86518984.html.

United Nations Population Fund Viet Nam. Ageing in viet nam. UNFPA Viet Nam thematic brief, 2021. URL https://vietnam.unfpa.org/en/topics/ageing-0. Accessed 14 October 2026.

World Health Organization. Who launches global effort to halve medication-related errors in 5 years. WHO news release, 2017. URL https://www.who.int/news/item/29-03-2017-who-launches-global-effort-to-halve-medication-related-errors-in-5-years. Accessed 14 October 2026.

World Health Organization. Health data as a global public good: A call for health data governance. WHO policy article, 2021a. URL https://www.who.int/news-room/articles-detail/health-data-as-a-global-public-good-a-call-for-health-data-governance-30-september. Accessed 14 October 2026.

World Health Organization. *Global Strategy on Digital Health 2020–2025*. World Health Organization, Geneva, 2021b. ISBN 978-92-4-002092-4. URL https://www.who.int/publications/i/item/9789240020924.

World Health Organization. Patient safety. WHO Fact Sheet, 2023. URL https://www.who.int/news-room/fact-sheets/detail/patient-safety. Accessed 14 October 2026.

World Health Organization Alliance for Health Policy and Systems Research. Harnessing digital transformation to improve health. Thematic programme overview, 2024. URL https://ahpsr.who.int/what-we-do/thematic-areas-of-focus/harnessing-digital-transformation-to-improve-health. Accessed 14 October 2026.